板子

数学

组合数学

基本板子

```
struct combine {
    typedef long long LL;
    const int N = 2e5 + 10, mod = 1e9 + 7;
    int inv[N], f[N], finv[N];
    void init(int n) {
       inv[0] = f[0] = finv[0] = inv[1] = f[1] = finv[1] = 1;
       for (int i = 2; i <= n; i++) {
           f[i] = 1LL * f[i - 1] * i % mod;
           inv[i] = 1LL * (mod - mod / i) * inv[mod % i] % mod;
           finv[i] = 1LL * finv[i - 1] * inv[i] % mod;
       }
    }
    int C(int a, int b) {
        if (a < 0 || b < 0 || a < b) return 0;
        return 1LL * f[a] * finv[b] % mod * finv[a - b] % mod;
    }
}
```

隔板法经典问题

求 $x_1 + x_2 + \ldots + x_n = m$ 解的个数。

1. $\forall i, x_i \geq 1$。

   问题可以看成 m 个求划分成 n 组，m 个球形成了 $m-1$ 个缝隙，从中选 $n-1$。

   故方案数等价于：$\binom{m-1}{n-1}$。

2. $\forall i, x_i \geq 0$。

   可以让方程两边加上 n，问题 $\iff$ $x_1 + 1 + x_2 + 1 + \ldots + x_n + 1 = m + n$。

   令 $x'_i = x_i + 1$，则有 $x'_1 + x'_2 + \ldots + x'_n = m + n$，问题变为第一种情况。

   故方案数为：$\binom{m+n-1}{n-1}$。

3. $\forall i, x_i \geq a_i$。

   两边同时减去 $\sum_i a_i$，问题 $\iff$ $x_1 - a_1 + x_2 - a_2 + \ldots + x_n - a_n = m - \sum_{i=1}^{n} a_i$

   令 $x'_i = x_i - a_i$，则有 $x'_1 + x'_2 + \ldots + x'_n = m - \sum_{i=1}^{n} a_i$，其中 $x'_i \geq 0$，转为第二种，然后即可求解，

   故方案数为：$\binom{m - \sum_{i=1}^{n} a_i + n - 1}{n-1}$。

4. $\forall i, 0 \leq x_i \leq y$。

考虑容斥，记 $f(x)$ 表示满足 $x_i \geq 0$ 这一条件解的个数，$g(x)$ 表示满足至少存在一个 $x_i > y$ 解的个数。

发现 $g(x)$ 可以用容斥求解，故方案数如下：
$\sum_{j=0}^{n} \binom{n}{j} \times \binom{m - j \times (y+1) + n - 1}{n-1} \times (-1)^j$。

错位排列

对于 $1 \backsim n$ 的排列 P，如果满足 $P_i \neq i$，则称 P 是 n 的错位排列。

$0, 1, 2, 9, 44, 265, \ldots$

递推式：$D_n = (n-1)(D_{n-1} + D_{n-2})$

公式：$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$

卡特兰数

常用于以下问题：

1. 栈的出栈序列
2. 从左上角走到右上角，每次只能向右或者向上走，不走到对角线上方
3. 圆上选 $2n$ 个点，让这些点对相连的 n 条线段互不相交
4. 对角线不相交，将一个凸多边形区域划分为三角形区域
5. n 个结点可以构造多少个不同的二叉树
6. 给 n 个 $+1$，和 n 个 -1，有多种排列是得任意前缀都 ≥ 0

$1, 1, 2, 5, 14, 42, 132 \ldots\ldots$

公式：$H_n = \binom{2n}{n} - \binom{2n}{n-1}$

递推：

- $H_n = \sum_{i=1}^{n} H_{i-1} H_{n-i} \quad n \geq 2$
- $H_n = 1 \;\; n = 0, 1$

斯特林数

第二类斯特林数

把 n 个两两不同的元素，划分为 k 个互不区分的非空子集的方案

$f_{n,k} = f_{n-1,k-1} + k \times f_{n-1,k}$

第一类斯特林数

把 n 个两两不同的元素，划分为 k 个互不区分的非空轮换（圆排列）的方案

$f_{n,k} = f_{n-1,k-1} + (n-1) \times f_{n-1,k}$

数论

逆元

若 $ax \equiv 1 \pmod b$，且 a 与 b 互质，那么我们称 x 为 a 在 $mod\ b$ 意义下的逆元，记作 $x = a^{-1}$。

快速幂求逆元

要求模数是质数

```
LL ksm(LL a, LL b, LL mod) {
    LL res = 1;
    a %= mod;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

LL get_inv(LL a, LL b) {
    return ksm(a, b - 2, b);
}
```

exgcd求逆元

不要求模数是质数

```
LL exgcd(LL a, LL b, LL& x, LL& y) {
    if (!b) {
        x = 1, y = 0;
        return a;
    }
    LL d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

LL get_inv(LL a, LL b) {
    LL x, y;
    exgcd(a, b, x, y);
    return (x % b + b) % b;
}
```

同余

若 $a \equiv b \pmod m$，$b \equiv c \pmod m$，则 $a \equiv c \pmod m$。

若 $a \equiv b \pmod m$，$c \equiv d \pmod m$，则有：

- $a \pm c \equiv b \pm d \pmod m$。
- $a \times c \equiv b \times d \pmod m$。

若 $a \equiv b \pmod m$，则 $ak \equiv bk \pmod{mk}$。

若 $d \mid m$，则当 $a \equiv b \pmod{m}$ 成立时，有 $a \equiv b \pmod{d}$。

若 $a \equiv b \pmod{m_i}, i = 1, 2, 3, \ldots, k$，则 $a \equiv b \pmod{\operatorname{lcm}(m_1, m_2, m_3, \ldots, m_k)}$。

若 $d \mid a$，$d \mid b$，则当 $a \equiv b \pmod{m}$ 成立时，有 $\frac{a}{d} \equiv \frac{b}{d} \pmod{\frac{m}{\gcd(m,d)}}$。

若 $a \equiv b \pmod{m}$，则 $\gcd(a, m) = \gcd(b, m)$。

欧拉定理

若 $\gcd(a, m) = 1$，则 $a^{\phi(m)} \equiv 1 \ (mod\ m)$。

推论：若 $\gcd(a, m) = 1$，则 $a^b \equiv a^{\phi(m)} \mod m$。

阶

满足 $a^n \equiv 1 \mod m$ 的最小正整数 n 称为 a 模 m 的阶，记作 $ord_m(a)$。

- 若 $a^n \equiv 1 \mod m$，则 $ord_m(a) \mid n$。

阶的求法：

```
int primes[N], mip[N], idx;
void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        if (!mip[i]) mip[i] = i, primes[idx++] = i;
        for (int j = 0; primes[j] <= n / i; j++) {
            mip[i * primes[j]] = primes[j];
            if (i % primes[j] == 0) break;
        }
    }
}
int ksm(int a, int b, int p) {
    int res = 1;
    while (b) {
        if (b & 1) res = 1LL * res * a % p;
        a = 1LL * a * a % p;
    }
    return res;
}

int calc(int a, int m) {
    int phi = m - 1;
    std::vector<int> pf;
    for (int i = 0; primes[i] <= phi / primes[i]; i++) {
        if (phi % primes[i] == 0) {
            pf.push_back(primes[i]);
            while (phi % primes[i] == 0) phi /= primes[i];
        }
    }
    if (phi > 1) pf.push_back(phi);
    int d = m - 1;
    for (auto &c : pf) {
        while (d % c == 0 && ksm(a, d / c, m) == 1) {
            d /= c;
        }
    }
    return d;
```

```
}
```

欧拉降幂

$$a^b \equiv \begin{cases} a^{b \bmod \phi(p)} & \gcd(a,p)=1 \\ a^b & \gcd(a,p)\neq 1, b<\phi(p) \\ a^{b \bmod \phi(p)+\phi(p)} & \gcd(a,p)\neq 1, b\geq \phi(p) \end{cases} \pmod p$$

费马小定理

设 p 为质数，且 p 不能整除 a，则 $a^{p-1} \equiv 1\ (mod\ p)$。

故此时 a 在模 p 意义下的逆元就为 a^{p-2}。

裴蜀定理

设 a, b 是不全为零的整数，$\gcd(a,b)=d$，那么对于任意的整数 x, y，一定有 $d \mid ax+by$，且一定存在整数 x, y 使得 $ax+by=d$ 成立。

n 个数的裴蜀定理：设 $a_1, a_2, a_3, \ldots, a_n$ 为 n 个不全为零的整数，$\gcd(a_1, a_2, a_3, \ldots, a_n)=d$，则一定存在一组整数解 $x_1, x_2, x_3, \ldots, x_n$ 使得 $x_1a_1+x_2a_2+x_3a_3+\ldots+x_na_n=d$。

一般的对于 $x_1a_1+x_2a_2+x3a_3+\ldots+x_na_n=m$ 有解，当且仅当 $\gcd(a_1, a_2, a_3, \ldots, a_n) \mid m$。

exgcd

```
template<typename T>
T exgcd(T a, T b, T &x, T &y) {
    if (!b) {
        x = 1, y = 0;
        return a;
    }
    T d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

几个 tips：

令 $(a,b)=\gcd(a,b)$

1. 对于 $ax+by$ 其能表示的所有整数形式必然为：$k(a,b)$，即 $ax+by=k(a,b)$。
2. 对于 $ax+by=c$，其有解的充要条件是：$(a,b) \mid c$。

   令 $d=\gcd(a,b)$，其所表示的通解形式如下：

   $$x=\frac{c}{d}x^*+k\frac{b}{d}$$
   $$y=\frac{c}{d}y^*-k\frac{a}{d}$$

   注意通过 exgcd 解出来的特解 x^*, y^* 一定是满足所有解里面，绝对值最小的。

   求最小非负整数解。

   $x_{\min}=(x_0\%\frac{b}{d}+\frac{b}{d})\%\frac{b}{d}$。

   $y_{\min}=(y_0\%\frac{a}{d}+\frac{a}{d})\%\frac{a}{d}$。

3. 对于一类同余的问题，要想到转成 exgcd 的表示形式，例如你设了 $n \equiv r\ (mod\ m)$，那么其实其等价于 $n = km + r$。
4. 对于求解 $ax + by + t$ 在模 m 意义下的最小值（非负整数），可以把问题转成：

   $\rightarrow$ 即求 $ax + by + t = k_1(a, b) + t$ 在模 m 意义下的最小值。

   $\rightarrow$ 可设 $k_1(a, b) + t \equiv r$，所以有 $k_1(a, b) + t = k_2m + r$，即 $r = k_1(a, b) - k_2m + t$，求其最小值

   $\rightarrow$ 由于 $k_1(a, b) - k_2m = k(a, b, m)$，所以 $r = k(a, b, m) + t$，显然其最小非负整数解就为：$t\%(a, b, m)$。

---

excrt

```
template<typename T>
void merge(T &a, T &b, T c, T d) {
    //合并x = kb + a 和 x = kd + c
    if (a == -1 && b == -1) return;
    T x, y;
    T g = exgcd(b, d, x, y);
    if ((c - a) % g) {
        a = b = -1;
        return;
    }
    d /= g;
    T t0 = ((c - a) / g) % d * x % d);
    if (t0 < 0) t0 += d;
    a = b * t0 + a;
    b = b * d;
}
```

常见数论函数

令 $n = p_1^{k_1}p_2^{k_2}...p_m^{k_m}$

1. 正约数个数：$d(n) = (p_1 + 1)(p_2 + 1)...(p_m + 1)$
2. 正约数个数和：
   $\sum_{d|n} d = (p_1^0 + p_1^1 + ... + p_1^{k_1})(p_2^0 + p_2^1 + ... + p_2^{k_2})...(p_m^0 + p_m^1 + ... + p_m^{k_m})$
3. 欧拉函数：$\phi(n) = n(1 - \frac{1}{p_1})(1 - \frac{1}{p_2})....(1 - \frac{1}{p_m})$

Pollard_Rho

```
typedef long long LL;
typedef __int128 i128;
LL gcd(LL a, LL b) {
  if (b == 0) return a;
  return gcd(b, a % b);
}

LL quick_pow(LL x, LL p, LL mod) {  // 快速幂
  LL ans = 1;
  while (p) {
```

```
    if (p & 1) ans = (i128)ans * x % mod;
    x = (i128)x * x % mod;
    p >>= 1;
  }
  return ans;
}

bool Miller_Rabin(LL n) {  // 判断素数
  if (n == 2) return true;
  if (n <= 1 || n % 2 == 0) return false;
  LL base[] = {2, 325, 9375, 28178, 450775, 9780524, 1795265022};
  LL u = n - 1, k = 0;
  while (u % 2 == 0) u /= 2, k++;
  for (auto x : base) {
      if (x % n == 0) continue;
      LL v = quick_pow(x, u, n);
      if (v == 1 || v == n - 1) continue;
      for (int j = 1; j <= k; j++) {
          LL last = v;
          v = (i128)v * v % n;
          if (v == 1) {
              if (last != n - 1) return false;
              break;
          }
      }
      if (v != 1) return false;
  }
    return true;
}

LL Pollard_Rho(LL n) {
  static std::mt19937_64
sj(std::chrono::steady_clock::now().time_since_epoch().count());
   std::uniform_int_distribution<LL> u0(1, n - 1);
   LL c = u0(sj);
   LL x = 0, y = 0, s = 1;
   for (int k = 1; ; k <<= 1, y = x, s = 1) {
       for (int i = 1; i <= k; i++) {
           x = ((i128)x * x + c) % n;
           s = (i128)s * std::abs(x - y) % n;
           if (i % 127 == 0) {
               LL d = gcd(s, n);
               if (d > 1) return d;
           }
       }
       LL d = gcd(s, n);
       if (d > 1) return d;
   }
    return n;
}

void fac(LL x, std::vector<LL> &ans) {
    //注意fac求出来的质因数是可能有重复的，最后求完ans要记得对ans去重
  if (x == 1) return;
  if (Miller_Rabin(x)) {              // 如果x为质数
    ans.push_back(x);  // 更新答案
```

```
    return;
  }
  LL p = x;
  while (p == x) p = Pollard_Rho(x);  // 使用该算法
  fac(p, ans), fac(x / p, ans);  // 继续向下分解x / p 和 p
}

//求最大的质因子
void get_maxfac(LL x, LL &ans) {
    if (x < 2) return ;
    if (Miller_Rabin(x)) {
        ans = std::max(ans, x);
        return ;
    }
    LL p = x;
    while (p == x) p = Pollard_Rho(p);
    get_maxfac(p, ans), get_maxfac(x / p , ans);
}

//求最小的质因子
void get_minfac(LL x, LL &ans) {
    if (x < 2) return ;
    if (Miller_Rabin(x)) {
        ans = std::min(ans, x);
        return ;
    }
    LL p = x;
    while (p == x) p = Pollard_Rho(p);
    get_minfac(p, ans), get_minfac(x / p , ans);
}
```

矩阵

```
void mul(int a[][N], int b[][N], int c[][N]) {
    static int g[N][N];
    std::memset(g, 0, sizeof g);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                g[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    std::memcpy(c, g, sizeof g);
}
struct Matrix {
    std::vector<std::vector<int>> mat;
    int n, m;
    Matrix(int _n, int _m) : n(_n), m(_m) {
        mat = std::vector<std::vector<int>>(n + 1, std::vector<int>(m + 1, 0));
    }
    void unit() {
```

```
        assert(n == m);
        for (int i = 1; i <= n; i++) {
            mat[i][i] = 1;
        }
    }
    Matrix operator*(const Matrix &rhs) const {
        assert(m == rhs.n);
        int _n = n, _m = rhs.m;
        Matrix tmp(_n, _m);
        for (int i = 1; i <= _n; i++) {
            for (int j = 1; j <= _m; j++) {
                for (int k = 1; k <= m; k++) {
                    tmp.mat[i][j] = ((tmp.mat[i][j] + 1LL * this->mat[i][k] *
rhs.mat[k][j] % mod) % mod + mod) % mod;
                }
            }
        }
        return tmp;
    }
};

struct Matrix {
    int c[4][4];
    Matrix() {std::memset(c, 0, sizeof c);}
    void init() {for (int i = 0; i < 4; i++) c[i][i] = 1;}
    void clear() {memset(c, 0, sizeof c);}
};

inline void add(int &x, int y) {
    x += y;
    if (x >= mod) x -= mod;
}

Matrix operator * (const Matrix &a, const Matrix &b) {
    Matrix g;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            for (int k = 0; k < 4; k++) {
                add(g.c[i][j], 1LL * a.c[i][k] * b.c[k][j] % mod);
            }
        }
    }
    return g;
};
Matrix operator + (const Matrix &a, const Matrix &b) {
    Matrix g;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            g.c[i][j] = (a.c[i][j] + b.c[i][j]) % mod;
        }
    }
    return g;
};
template <typename T>
struct Matrix {
    std::vector<std::vector<T>> mat;
```

```
    int n_, m_;
    Matrix(int n, int m) : n_(n), m_(m) {
        mat = std::vector<std::vector<T>>(n, std::vector<T>(m, 0));
    }
    void unit() {
        assert(n_ == m_);
        for (int i = 0; i < n_; i++) {
            mat[i][i] = 1;
        }
    }
    Matrix operator*(const Matrix &rhs) const {
        assert(m_ == rhs.n_);
        int new_n = n_, new_m = rhs.m_;
        Matrix tmp(new_n, new_m);
        for (int i = 0; i < new_n; i++) {
            for (int j = 0; j < new_m; j++) {
                for (int k = 0; k < m_; k++) {
                    tmp.mat[i][j] = (tmp.mat[i][j] + this->mat[i][k] * rhs.mat[k]
[j]);
                }
            }
        }
        return tmp;
    }
};

template <typename T1, typename T2>
Matrix<T1> MatrixPower(Matrix<T1> a, T2 b) {
  Matrix<T1> res(a.n_, a.m_);
  res.unit();
  while (b) {
    if (b & 1) res = res * a;
    a = a * a;
    b >>= 1;
  }
  return res;
}
```

高斯消元

```
const int N = 110;
const double eps = 1e-10
int a[N][N], b[N]; //b是系数矩阵
void gauss() {
    int l = 1; //当且消到第几行
    for (int i = 1; i <= n; i++) { //枚举列
        //选这一列选一个绝对值最大的换上来
        for (int j = l + 1; j <= n; j++) {
            if (abs(a[j][i]) > abs(a[l][i])) {
                for (int k = i; k <= n; k++) swap(a[j][k], a[l][k]);
                swap(b[j], b[l]);
            }
        }
```

```
        if (abs(a[l][i]) < eps) continue; // 说明这一列全是0了，不用消了
        //把这一列的所有行除了第l行全变成0
        for (int j = 1; j <= n; j++) {
            if (j != l && abs(a[j][i]) >= eps) {
                double delta = a[j][i] / a[l][i];
                for (int k = i; k <= n; k++) {
                    a[j][k] -= a[l][k] * detal;
                }
                b[j] -= delta * b[l];
            }
        }
        l++;
    }
    for (int i = l; i <= n; i++) {
        //无解
        if (abs(b[i]) >= eps) {
            cout << "-1\n";
            return;
        }
    }
    //我们一共会得到l - 1 个元的答案。(其中 l - 1 为矩阵的秩)
    //其中自由元个数为: n - (l - 1) = n - l + 1 个
    if (l <= n) {
        //无穷多解，存在自由元了
        cout << "2\n";
    } else {
        //此时必然化为了最简阶梯状
        for (int i = 1; i <= n; i++) {
            b[i] /= a[i][i];
        }
    }
}

//异或的高斯消元

void gauss() {
    int l = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = l + 1; j <= n; j++) {
            if (a[j][i]) {
                for (int k = i; k <= n; k++)
                    swap(a[j][k], a[l][k]);
                swap(b[j], b[l]);
                break;
            }
        }
        if (!a[l][i]) continue;
        for (int j = 1; j <= n; j++) {
            if (j != l && a[j][i]) {
                for (int k = i; k <= n; k++) {
                    a[j][k] ^= a[l][k];
                }
                b[j] ^= b[l];
            }
        }
        l++;
```

```
    }
    for (int i = l; i <= n; l++) {
        if (b[i]) {
            cout << -1 << "\n";
            return;
        }
    }

}
```

线性基

```
typedef long long LL;
const int B = 63;
struct Linear_basis {
    LL p[B];
    int sz, n;

    void init(int n) {
        for (int i = 0; i <= B - 1; i++) {
            p[i] = 0;
        }
        sz = 0;
        this->n = n;
    }
    bool insert(LL x) {
        for (int i = B - 1; i >= 0; i--) {
            if (x >> i & 1) {
                if (!p[i]) {
                    p[i] = x;
                    return true;
                }
                x ^= p[i];
            }
        }
        return false;
    }
    LL querymax() {
        LL res = 0;
        for (int i = B - 1; i >= 0; i--) {
            res = std::max(res, res ^ p[i]);
        }
        return res;
    }
    LL querymin() {
        if (sz < n) return 0;
        for (int i = 0; i <= B - 1; i++) {
            if (p[i]) return p[i];
        }
    }
    LL query_max(LL x) {
        for (int i = B - 1; i >= 0; i--) {
```

```
            x = std::max(x, x ^ p[i]);
        }
        return x;
    }
    LL query_min(LL x) {
        for (int i = B - 1; i >= 0; i--) {
            x = std::min(x, x ^ p[i]);
        }
        return x;
    }
    void rebuild() {
        for (int i = B - 1; i >= 0; i--) {
            for (int j = i - 1; j >= 0; j--) {
                if (p[i] >> j & 1) p[i] ^= p[j];
            }
        }
        for (int i = 0; i <= B - 1; i++) {
            if (p[i]) p[sz++] = p[i];
        }
    }
    //第k大
    LL query_kmax(LL k) {
        if (sz < n && k == (1LL << sz)) return 0;
        LL res = 0;
        for (int i = sz - 1; i >= 0; i--) {
            if (k >> i & 1) res ^= p[i];
        }
        return res;
    }
    //第k小
    LL query_kmin(LL k) {
        if (sz < n) k--;
        if (k >= (1LL << sz)) return -1;
        if (!k) return 0;
        LL res = 0;
        for (int i = 0; i < sz; i++) {
            if (k >> i & 1) res ^= p[i];
        }
        return res;
    }
};
```

带时间戳的线性基，维护区间xor

```
typedef long long LL;

const int B = 63;
struct Linear_basis {
    LL p[B];
    int t[B];
    void insert(LL x, int _t) {
        for (int i = B - 1; i >= 0; i--) {
            if (x >> i & 1) {
                if (!p[i]) {
                    p[i] = x;
                    t[i] = _t;
```

```
                    return;
                } else {
                    if (t[i] < _t) {
                        swap(p[i], x);
                        swap(t[i], _t);
                    }
                    x ^= p[i];
                }
            }
        }
    }
    LL query_max(LL x, int _t) {
        for (int i = B - 1; i >= 0; i--) {
            if (t[i] >= _t) x = std::max(x, x ^ p[i]);
        }
        return x;
    }
    void ins(const Linear_basis &a) {
        for (int i = B - 1; i >= 0; i--) {
            if (a.p[i]) insert(a.p[i], a.t[i]);
        }
    }
};
```

莫比乌斯反演

$\varepsilon(n) = [n = 1]$

$id_k(n) = n^k$，$id(n) = n$

$1(n) = 1$

$\varphi(i) = \sum_{i=1}^{n}[gcd(i, n) = 1]$

$\sigma_k(n) = \sum_{d|n} d^k$

$$\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & n \text{ 有平方因子} \\ (-1)^k & n = p_1^{x_1} p_2^{x_2} \dots p_k^{x_k} \end{cases}$$

若 $\gcd(a_1, a_2, \dots, a_n) = x$，且 $1 \le a_i \le k$，现在要求 $\sum_{x|d}[\gcd(a_1, a_2, \dots, a_n) = d] = \lfloor \frac{k}{x} \rfloor^n$

。

如果要求 gcd 恰好为多少的，我们可以先求 gcd 为... 倍数的，然后容斥减一下。

迪利克雷卷积：

$(f * g)(n) = \sum_{d|n} f(d) g(\frac{n}{d})$

若 f, g 都为积性函数，那么 $f * g$ 也为积性函数

- $\varepsilon = \mu * 1$

  即 $[n = 1] = \sum_{d|n} \mu(d)$

- $id = \varphi * 1$

  即 $n = \sum_{d|n} \varphi(d)$
- $\sigma_k = id * 1$

  即 $\sigma_k(n) = \sum_{d|n} d^k$
- 第一种莫反（约数形式）

  $f = g * 1 \iff g = f * \mu$

  设 $f(n) = \sum_{d|n} g(d)$

  则 $g(n) = \sum_{d|n} f(d)\mu(\frac{n}{d})$
- 第二种莫反（倍数形式）

  设 $f(n) = \sum_{n|d} g(d)$

  则 $g(n) = \sum_{n|d} f(d)\mu(\frac{d}{n})$
- $\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{x|d}[\gcd(i,j)=d] = \lfloor\frac{n}{x}\rfloor\lfloor\frac{m}{x}\rfloor$
- $d(ij) = \sum_{x|i}\sum_{y|j}[\gcd(x,y)=1]$
- $\sum_{d|n} = \frac{\mu(d)}{d} = \frac{\phi(n)}{n}$

反演技巧：

1. 改变求和指标：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[gcd(i,j)=k]$$
$$=\sum_{i=1}^{n}\sum_{j=1}^{m}[gcd(\frac{i}{k},\frac{j}{k})=1]$$
$$=\sum_{i=1}^{\lfloor\frac{n}{k}\rfloor}\sum_{j=1}^{\lfloor\frac{m}{k}\rfloor}[gcd(i,j)=1]$$

2. 经典莫反变形，利用 $[n=1] = \sum_{d|n}\mu(d)$

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d|\gcd(i,j)}\mu(d)$$

1. 将形如 $\sum_{i=1}^{n}\sum_{j=1}^{m} f(\gcd(i,j))$ 的式子化为含有 $[\gcd(i,j)=1]$ 的式子。

$$\sum_{i=1}^{n}\sum_{j=1}^{m} f(\gcd(i,j)) =$$
$$\sum_{d=1}^{n}\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}\sum_{j=1}^{\lfloor\frac{m}{d}\rfloor} f(d)[\gcd(i,j)=1]$$

1. 将 $\sum_{i=1}^{n}\sum_{j=1}^{n} f(i,j)$ 和 $\sum_{i=1}^{n}\sum_{j=i}^{n} f(i,j)$ 相互转化。

$$\sum_{i=1}^{n}\sum_{j=i}^{n} f(i,j) = \frac{\sum_{i=1}^{n}\sum_{j=1}^{n} f(i,j) + \sum_{i=1}^{n} f(i,i)}{2}$$

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[gcd(i,j)=k]$$

$$=\sum_{i=1}^{n}\sum_{j=1}^{m}[gcd(\frac{i}{k},\frac{j}{k})=1]$$

$$=\sum_{i=1}^{\lfloor\frac{n}{k}\rfloor}\sum_{j=1}^{\lfloor\frac{m}{k}\rfloor}[gcd(i,j)=1]$$

$$=\sum_{i=1}^{\lfloor\frac{n}{k}\rfloor}\sum_{j=1}^{\lfloor\frac{m}{k}\rfloor}\sum_{d|gcd(i,j)}\mu(d)$$

$$=\sum_{d=1}^{min(\lfloor\frac{n}{k}\rfloor,\lfloor\frac{m}{k}\rfloor)}\mu(d)\sum_{i=1}^{\lfloor\frac{n}{k}\rfloor}[d|i]\sum_{j=1}^{\lfloor\frac{m}{k}\rfloor}[d|j]$$

$$=\sum_{d=1}^{min(\lfloor\frac{n}{k}\rfloor,\lfloor\frac{m}{k}\rfloor)}\mu(d)\lfloor\frac{n}{dk}\rfloor\lfloor\frac{m}{dk}\rfloor$$

```
typedef long long LL;
const int N = 5e4 + 10;
int primes[N], mu[N], s[N], idx;
bool st[N];

void init(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!st[i]) primes[idx++] = i, mu[i] = -1;
        for (int j = 0; primes[j] <= n / i; j++) {
            st[primes[j] * i] = true;
            mu[i * primes[j]] = -mu[i];
            if (i % primes[j] == 0) {
                mu[i * primes[j]] = 0;
                break;
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        s[i] = s[i - 1] + mu[i];
    }
}

int f(int n, int m, int k) {
    n /= k, m /= k;
    LL ans = 0;
    int up = min(n, m);
    for (int l = 1, r; l <= up; l = r + 1) {
        r = std::min({n / (n / l), m / (m / l),  up});
        ans += 1LL * (s[r] - s[l - 1]) * (n / l) * (m / l);
    }
    return ans;
}
```

$\sum_{i=1}^{n}\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}\varphi(i)=\sum_{i=1}^{n}\sum_{d|i}\varphi(d)=\sum_{i=1}^{n}i=\frac{n\times(n+1)}{2}$

概率期望

几大概率类型以及公式：

- 古典概型：若总的基本事件个数为 n，A 发生的基本事件个数为 m，则 A 发生的概率 $P(A)=\frac{m}{n}$
- 条件概率：事件 A 在另一个事件 B 已经发生的条件下发生的概率为 $P(A|B)=\frac{P(AB)}{P(B)}$
- 全概率公式：

  对于一个完备的事件组：$B_1,B_2,B_3,.....,B_n$，$\forall i,j,B_i\cap B_j=0$ (即两两互斥)

  且 $B_1\cup B_2\cup.....\cup B_n=\Omega$，其中 Ω 为样本空间。

  那么对于任意一个事件 A，其发生的概率为：$P(A)=\sum_{i=1}^{n}P(B_i)P(A|B_i)$
- 贝叶斯公式：

  该公式的作用是已知一个大事件 A 已经发生的概率的情况下，问你在样本空间Ω的一个划分：$B_1,B_2,...,B_n$

  这些事件在 A 发生的概率：$P(B_i|A)=\frac{P(B_i)P(A|B_i)}{\sum_{i=1}^{n}P(B_i)P(A|B_i)}$

几个常见的概率分布：

- 两点分布：进行一次伯努利试验里面结果只有两种发生和不发生，其概率分别为 P，和 $1-P$

  期望：$E(x)=p$
- 二项分布：做 n 次伯努利试验，其中有 k 次成功的概率：$P\{x=k\}=C_n^kp^k(1-p)^{n-k}$

  期望：$E(x)=np$
- 几何分布：做 n 次伯努利试验，其中当第 k 次试验才成功，意味着前 $k-1$ 次全失败的概率：$P(k)=(1-p)^{k-1}p$

  期望：$E(x)=\dfrac{1}{p}$

期望定义：

对于一个离散的随机变量：x，其可能的取值为：$x_1,x_2,...,x_n$，其发生的概率分别为：$P_1,P_2,...,P_n$

那么期望值：$E(x)=\sum_{i=n}^{n}p_i*x_i$

期望的性质：

- $E(c)=c$
- $E(cx)=cE(x)$
- $E(ax+by)=aE(x)+bE(y)$
- 若 x,y 独立，则 $E(xy)=E(x)E(y)$
- 期望的线性性质：$E(\sum x)=\sum E(x)$

全期望公式：$E(Y)=\sum_i P(X=x_i)E(Y|X=x_i)$

期望 dp 和概率 dp : 一般期望 dp 的状态定义为 $dp[i]$ 表示从 i 到目标状态的期望

一般概率 dp 的状态定义为 $dp[i]$ 表示从初始状态到 i 的概率

当然这都不是绝对的，期望 dp 也可以正推，其主要看构成的拓扑图是否唯一，之所以倒推是其可以保证终点唯一

这样不用维护一个额外的概率数组

当然此类 dp 还是按照正常的 dp，哪个状态时已知就从哪边开始推

概率dp: $dp[u] = p * dp[v]$ ，其中一般表示 $v->u$，且 p 为从 $v->u$ 的概率

期望dp: $dp[u] = p * (dp[v] + w)$，其中一搬表示 $u->v$，且p 为 $u->v$ 的概率

当然有一种更加直观的，就是我看看 $dp[u]$ 的实际意义即这个事件可以分解成几个事件的和，从而得到转移

总结来说就时期望 dp 转移的时候需要 $+$ 上边权的代价，概率 dp 则不用

整除分块

$$\sum_{i=1}^{n} \lfloor \frac{n}{i} \rfloor$$

$i \in [l, n/(n/l)]$ 是一段相同的值

```
long long calc(long long st, long long ed, long long n) {
    long long ans = 0;
    ed = std::min(ed, n);
    for (long long l = st, r; l <= ed; l = r + 1) {
        r = std::min(ed, n / (n / l));
        ans += (r - l + 1) * (n / l);
    }
    return ans;
}
```

三分

```
int main()
{
    //函数的凹凸性质可以使用打表制作图像，或者求导判断

    //三分本质：每次消去不可能存在正确答案的区间

    int l = 0, r = 3.14159265 / 2;//取值范围
    //浮点型凹函数
    for(int i = 1;i <= 300;i++) //浮点数直接固定次数
    {
        double lmid = l + (r - l) / 3, rmid = r - (r - l) / 3;
        if (f(lmid) > f(rmid))l = lmid;
        else r = rmid;
    }
    //浮点型凸函数
    for(int i = 1;i <= 300;i++) //浮点数直接固定次数
    {
        double lmid = l + (r - l) / 3, rmid = r - (r - l) / 3;
        if (f(lmid) < f(rmid))l = lmid;//>改为<
        else r = rmid;
    }
    //整型凹函数
    while (l < r)
```

```
    {
        int lmid = l + (r - l) / 3, rmid = r - (r - l) / 3;
        if (f(lmid) > f(rmid))l = lmid + 1;
        else r = rmid - 1;
    }
    //整型凸函数
    while (l < r)
    {
        int lmid = l + (r - l) / 3, rmid = r - (r - l) / 3;
        if (f(lmid) < f(rmid))l = lmid + 1;
        else r = rmid - 1;
    }
}
```

康托展开

康托展开用于求 $1 \backsim n$ 的排列字典序的排名。

序列 $a_1, a_2, \ldots, a_n$ 的 $rank = \sum_i^{n-1} A_i \times (n-i)!$

其中 A_i 表示在 $i+1 \backsim n$ 中比 a_i 小的数的个数。

```
LL f[N], A[N], a[N];
//f[i] 表示 i的阶乘。
LL cantor(int *p, int n) {
    LL ans = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) {
            if (a[j] < a[i]) A[i]++;
        }
    }
    for (int i = 1; i < n; i++) {
        ans += A[i] * f[n - i];
    }
    return ans + 1;
}
```

逆康托展开：

求 $rank = k$ 的排列是啥。

模板1：

```
LL f[N], p[N], A[N];
void decantor(LL x, int n) {
    x--;
    iota(rest.begin(), rest.end(), 1); // 将rest初始化为1,2,...,n
    for (int i = 1; i <= n; ++i)
    {
        A[i] = x / fact[n - i];
        x %= fact[n - i];
    }
    for (int i = 1; i <= n; ++i)
    {
        p[i] = rest[A[i]];
        rest.erase(lower_bound(rest.begin(), rest.end(), p[i]));
```

```
    }
}
```

模板2：

```
LL f[N], p[N];
bool st[N];
void decantor(LL x, int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (!st[j]) {
                if (x > f[n - i]) x -= f[n - i];
                else {
                    st[j] = true;
                    p[i] = j;
                    break;
                }
            }
        }
    }
}
```

求 $1 \backsim n$ 的排列中字典序第 k 小中的第 x 个位置的数是啥。

一般通过范围：$1 \le n, k \le 10^9$

我们发现其实在前面很多位置 $f[n-i] > k$ 的，即前面很多位置都是按 $1, 2, 3...$ 这样的顺序排好的

我们只要考虑啥时候 $k >= f[t]$，即只要考虑后 t 位即可。

生成树计数

- 对于一个 n 个点的无向完全图，其生成树个数为 n^{n-2}。
- 对于一个 n 个点的无向完全图，每条边在其生成树中的贡献均为：$\frac{n^{n-2} \times (n-1)}{\frac{n \times (n-1)}{2}} = 2 \times n^{n-3}$。

数据结构

bitset

```
二进制常用函数：
__builtin_popcount(x); //二进制里面1的个数(unsigned int类型)
__builtin_popcountll(x); //二进制里面1的个数(long long)
__builtin_parity(x); //1的个数的奇偶性
__ctz(x); //x二进制末尾0的个数
__clz(x); //x二进制开头0的个数
时间复杂度都是O(1)
---------------------------------------

时间复杂度都是 N / w  (w一般为64)
bitset<N> a
a[i] = 0,a[i] = 1
```

```
a.any(); //是否存在一个1
a.none(); //是否全是0
a.count(); //1的个数
&, |, ^, ~, <<, >>
a.set(x); //把x这一位变成1
a.set(); //把a所有位置变成1
a.reset(x); //把x这一位变成0
a.reset(); //把a所有位置变成0
a.flip(x); //这一位取反
a.flip();  //所有位取反

a._Find_first(); //找到a的第一个1的位置
a._Find_next(i); //从i往后找下一个1的位置
for(int i = a._Find_first(); i != a.size();i = a._Find_next(i)){
    std::cout << i << ' ';
}
```

树状数组

```
template<typename T>
struct BIT {
    std::vector<T> c;
    int n;
    BIT(int n) : n(n - 1), c(n) {}
    void update(int x, int k) {
        assert(x != 0);
        for (int i = x; i <= n; i += i & -i) {
            c[i] += k;
        }
    }
    T ask(int x) {
        T res = 0;
        for (int i = x; i; i -= i & -i) {
            res += c[i];
        }
        return res;
    }
    T ask(int l, int r) {
        if (l > r) return T(0);
        return ask(r) - ask(l - 1);
    }
    T ask_k(T k) {
        T ans = 0;
        for (int i = 30; i >= 0; i--) {
            if (ans + (1 << i) <= n && c[ans + (1 << i)] < k) {
                ans += 1 << i;
                k -= c[ans];
            }
        }
        return ans + 1;
    }
};
```

并查集

```
struct DSU {
    int p[N], sz[N];
    void init(int n) {
        for (int i = 1; i <= n; i++) {
            p[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        return x == p[x] ? x : p[x] = find(p[x]);
    }
    bool merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y];
        p[y] = x;
        return true;
    }
    int size(int x) {
        return sz[find(x)];
    }
    bool same(int x, int y) {
        return find(x) == find(y);
    }
};

//带权并查集
int find(int x) {
    if (x == p[x]) return x;
    int fx = find(p[x]);
    d[x] += d[p[x]];
    return p[x] = fx;
}
```

线段树

基本板子

```
struct Info {
    int cnt, len;
};

struct Tag {
    int lazy;
    Tag() : lazy(-1) {}
    Tag(int x) : lazy(x) {}
};

Info operator+(const Info& a, const Info& b) {
    Info res;
    res.cnt = a.cnt + b.cnt;
    res.len = a.len + b.len;
```

```
    return res;
}

Info operator+(const Info& a, const Tag& v) {
    Info res = a;
    res.cnt = v.lazy * res.len;
    return res;
}

Tag operator+(const Tag& a, const Tag& b) {
    Tag res = b;
    return res;
}

const int N = 1e5 + 10;

Info init[N];


struct Seg_tree {
    Info info[4 * N];
    Tag tag[4 * N];

    void pull(int u) {
        info[u] = info[u << 1] + info[u << 1 | 1];
    }

    void apply(int u, const Tag& v) {
        info[u] = info[u] + v;
        tag[u] = tag[u] + v;
    }

    void push(int u) {
        if (tag[u].lazy != -1) {
            apply(u << 1, tag[u]);
            apply(u << 1 | 1, tag[u]);
            tag[u] = Tag();
        }
    }

    void build(int u, int l, int r, Info* init) {
        if (l == r) {
            info[u] = init[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(u << 1, l, mid, init);
        build(u << 1 | 1, mid + 1, r, init);
        pull(u);
    }

    void modify(int u, int l, int r, int ql, int qr, const Tag& v) {
        if (l >= ql && r <= qr) {
            apply(u, v);
            return;
        }
```

```
        push(u);
        int mid = (l + r) >> 1;
        if (ql <= mid) modify(u << 1, l, mid, ql, qr, v);
        if (qr > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, v);
        pull(u);
    }

    Info query(int u, int l, int r, int ql, int qr) {
        if (l >= ql && r <= qr) return info[u];
        push(u);
        int mid = (l + r) >> 1;
        if (qr <= mid) return query(u << 1, l, mid, ql, qr);
        if (ql > mid) return query(u << 1 | 1, mid + 1, r, ql, qr);
        else return query(u << 1, l, mid, ql, qr) + query(u << 1 | 1, mid + 1, r,
ql, qr);
    }
};

//线段树上二分，找第一个满足条件的点
int find(int u, int l, int r, int ql, int qr) {
    if (l >= ql && r <= qr && !check(u)) return -1;
    if (l == r) return l;
    int mid = (l + r) >> 1;
    int p = -1;
    if (ql <= mid) p = find(u << 1, l, mid, ql, qr);
    if (qr > mid && p == -1) p = find(u << 1, mid + 1, r, ql, qr);
    return p;
}

----
jly线段树板子
template<class Info, class Tag>
struct LazySegmentTree {
    int n;
    std::vector<Info> info;
    std::vector<Tag> tag;
    LazySegmentTree() : n(0) {}
    LazySegmentTree(int n_, Info v_ = Info()) {
        init(n_, v_);
    }
    template<class T>
    LazySegmentTree(std::vector<T> init_) {
        init(init_);
    }
    void init(int n_, Info v_ = Info()) {
        init(std::vector(n_, v_));
    }
    template<class T>
    void init(std::vector<T> init_) {
        n = init_.size();
        info.assign(4 << std::__lg(n), Info());
        tag.assign(4 << std::__lg(n), Tag());
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                info[p] = init_[l];
                return;
```

```
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m, r);
            pull(p);
        };
        build(1, 0, n);
    }
    void pull(int p) {
        info[p] = info[2 * p] + info[2 * p + 1];
    }
    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);
    }
    void push(int p) {
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag();
    }
    void modify(int p, int l, int r, int x, const Info &v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        if (x < m) {
            modify(2 * p, l, m, x, v);
        } else {
            modify(2 * p + 1, m, r, x, v);
        }
        pull(p);
    }
    void modify(int p, const Info &v) {
        modify(1, 0, n, p, v);
    }
    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= y || r <= x) {
            return Info();
        }
        if (l >= x && r <= y) {
            return info[p];
        }
        int m = (l + r) / 2;
        push(p);
        return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
    }
    Info rangeQuery(int l, int r) {
        return rangeQuery(1, 0, n, l, r);
    }
    void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
        if (l >= y || r <= x) {
            return;
        }
        if (l >= x && r <= y) {
```

```
            apply(p, v);
            return;
        }
        int m = (l + r) / 2;
        push(p);
        rangeApply(2 * p, l, m, x, y, v);
        rangeApply(2 * p + 1, m, r, x, y, v);
        pull(p);
    }
    void rangeApply(int l, int r, const Tag &v) {
        return rangeApply(1, 0, n, l, r, v);
    }
    template<class F>
    int findFirst(int p, int l, int r, int x, int y, F pred) {
        if (l >= y || r <= x || !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int m = (l + r) / 2;
        push(p);
        int res = findFirst(2 * p, l, m, x, y, pred);
        if (res == -1) {
            res = findFirst(2 * p + 1, m, r, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findFirst(int l, int r, F pred) {
        return findFirst(1, 0, n, l, r, pred);
    }
    template<class F>
    int findLast(int p, int l, int r, int x, int y, F pred) {
        if (l >= y || r <= x || !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int m = (l + r) / 2;
        push(p);
        int res = findLast(2 * p + 1, m, r, x, y, pred);
        if (res == -1) {
            res = findLast(2 * p, l, m, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findLast(int l, int r, F pred) {
        return findLast(1, 0, n, l, r, pred);
    }
};

constexpr int C = 1 << 18;
constexpr int inf = 1E9;
```

```
struct Tag {
    int add = 0;

    void apply(const Tag &t) & {
        add += t.add;
    }
};

struct Info {
    int max = 0;

    void apply(const Tag &t) & {
        max += t.add;
    }
};

Info operator+(const Info &a, const Info &b) {
    return {std::max(a.max, b.max)};
}

---

struct Tag {
    int add = 0;
    void apply(const Tag &t) {
        add += t.add;
    }
};

struct Info {
    int mx = 0;
    void apply(const Tag &t) {
        mx += t.add;
    }
};

Info operator + (const Info &a, const Info &b) {
    return {max(a.mx, b.mx)};
}

struct Segtree {
    int n;
    vector<Info> info;
    vector<Tag> tag;
    template<class T>
    Segtree(int n, const T &init) : n(n), info(4 * n), tag(4 * n) {
        build(1, 1, n, init);
    }
    void pull(int u) {
        info[u] = info[u << 1] + info[u << 1 | 1];
    }
    void apply(int u, const Tag &v) {
        info[u].apply(v);
        tag[u].apply(v);
    }
```

```
    void push(int u) {
        apply(u << 1, tag[u]);
        apply(u << 1 | 1, tag[u]);
        tag[u] = Tag();
    }
    template<class T>
    void build(int u, int l, int r, const T &init) {
        if (l == r) {
            info[u] = init[u];
            return;
        }
        int mid = (l + r) >> 1;
        build(u << 1, l, mid, init);
        build(u << 1 | 1, mid + 1, r, init);
        pull(u);
    }
    void modify(int u, int l, int r, int x, const Info &v) {
        if (l == r) {
            info[u] = v;
            return;
        }
        int mid = (l + r) >> 1;
        if (x <= mid) modify(u << 1, l, mid, x, v);
        else modify(u << 1 | 1, mid + 1, r, x, v);
        pull(u);
    }
    void modify(int u, int l, int r, int ql, int qr, const Tag &v) {
        if (l >= ql && r <= qr) {
            apply(u, v);
            return;
        }
        int mid = (l + r) >> 1;
        push(u);
        if (ql <= mid) modify(u << 1, l, mid, ql, qr, v);
        if (qr > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, v);
        pull(u);
    }
    Info query(int u, int l, int r, int ql, int qr) {
        if (l >= ql && r <= qr) return info[u];
        int mid = (l + r) >> 1;
        push(u);
        Info res;
        if (ql <= mid) res = res + query(u << 1, l, mid, ql, qr);
        if (qr > mid) res = res + query(u << 1 | 1, mid + 1, r, ql, qr);
        return res;
    }
    bool check(int u) {

    }
    int findr(int u, int l, int r, int ql, int qr) {
        if (l >= ql && r <= qr && !check(u)) return -1;
        if (l == r) return l;
        int mid = (l + r) >> 1;
        push(u);
        int p = -1;
        if (ql <= mid) p = findr(u << 1, l, mid, ql, qr);
```

```
        if (qr > mid && p == -1) p = findr(u << 1 | 1, mid + 1, r, ql, qr);
        return p;
    }
    int findl(int u, int l, int r, int ql, int qr) {
        if (l >= ql && r <= qr && !check(u)) return -1;
        if (l == r) return l;
        int mid = (l + r) >> 1;
        push(u);
        int p = -1;
        if (qr > mid) p = findl(u << 1 | 1, mid + 1, r, ql, qr);
        if (ql <= mid && p == -1) p = findl(u << 1, l, mid, ql, qr);
        return p;
    }
};



------
动态开点线段树:
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N = 1e5 + 10;
const LL INF = 1e18 + 5;
struct node {
    int l, r;
    LL c0, c1;
    int tag1, tag2;
} tr[70 * N];
int idx;

int creat(LL l, LL r) {
    tr[++idx] = {0, 0, r - l + 1, 0, -1, 0};
    return idx;
}
void pushup(int u) {
    tr[u].c0 = tr[tr[u].l].c0 + tr[tr[u].r].c0;
    tr[u].c1 = tr[tr[u].l].c1 + tr[tr[u].r].c1;
}
void setinfo(int u, int ty, LL len) {
    if (ty != 2) {
        tr[u].tag1 = ty;
        tr[u].tag2 = 0;
        tr[u].c0 = ty == 0 ? len : 0;
        tr[u].c1 = ty == 1 ? len : 0;
    } else {
        tr[u].tag2 ^= 1;
        swap(tr[u].c0, tr[u].c1);
    }
}
void pushdown(int u, LL l, LL r) {
    LL mid = (l + r) >> 1;
    if (!tr[u].l) tr[u].l = creat(l, mid);
    if (!tr[u].r) tr[u].r = creat(mid + 1, r);
    if (~tr[u].tag1) {
        setinfo(tr[u].l, tr[u].tag1, mid - l + 1);
```

```
        setinfo(tr[u].r, tr[u].tag1, r - mid);
        tr[u].tag1 = -1;
    }
    if (tr[u].tag2) {
        setinfo(tr[u].l, 2, mid - l + 1);
        setinfo(tr[u].r, 2, r - mid);
        tr[u].tag2 = 0;
    }
}

void update(int &u, LL l, LL r, LL pl, LL pr, int ty) {
    if (!u) u = creat(l, r);
    if (l >= pl && r <= pr) {
        setinfo(u, ty, r - l + 1);
        return ;
    }
    pushdown(u, l, r);
    LL mid = (l + r) >> 1;
    if (pl <= mid) update(tr[u].l, l, mid, pl, pr, ty);
    if (pr > mid) update(tr[u].r, mid + 1, r, pl, pr, ty);
    pushup(u);
}

LL ask(int &u, LL l, LL r) {
    if (!u) u = creat(l, r);
    if (l == r) return l;
    LL mid = (l + r) >> 1;
    pushdown(u, l, r);
    if (tr[tr[u].l].c0) return ask(tr[u].l, l, mid);
    else return ask(tr[u].r, mid + 1, r);
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    int n;
    cin >> n;
    int root = 0;
    while (n--) {
        int op;
        LL l, r;
        cin >> op >> l >> r;
        if (op == 1) {
            update(root, 1, INF, l, r, 1);
        } else if (op == 2) {
            update(root, 1, INF, l, r, 0);
        } else {
            update(root, 1, INF, l, r, 2);
        }
        cout << ask(root, 1, INF) << "\n";
    }

    return 0;
}
```

线段树优化建图

线段树优化建图：

常用于优化类似一个点 u 向区间 $[l, r]$ 所有点连边，或者区间 $[l, r]$ 所有点向点 u 连边。

思想是建立两颗线段树，分别为入边树和出边树。

采用动态开点线段树：

```
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
typedef pair<LL, int> PLI;
const int N = 1e5 + 10;
const LL INF = 1e18;
int lson[4 * N], rson[4 * N], tot;
vector<array<int, 2>> g[10 * N];
LL dis[10 * N];
bool st[10 * N];


void build(int &u, int l, int r, int type) {
    if (l == r) {
        u = l;
        return ;
    }
    u = ++tot;
    int mid = (l + r) >> 1;
    build(lson[u], l, mid, type);
    build(rson[u], mid + 1, r, type);
    if (type == 0) {
        g[u].push_back({lson[u], 0});
        g[u].push_back({rson[u], 0});
    } else {
        g[lson[u]].push_back({u, 0});
        g[rson[u]].push_back({u, 0});
    }
}

void change(int u, int l, int r, int L, int R, int v, int w, int type) {
    if (l >= L && r <= R) {
        if (type == 0) g[v].push_back({u, w});
        else g[u].push_back({v, w});
        return ;
    }
    int mid = (l + r) >> 1;
    if (L <= mid) change(lson[u], l, mid, L, R, v, w, type);
    if (R > mid) change(rson[u], mid + 1, r, L, R, v, w, type);
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    int n, q, s;
    cin >> n >> q >> s;
    int root1, root2;
```

```
    tot = n;
    build(root1, 1, n, 0); //入
    build(root2, 1, n, 1); //出
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int v, u, w;
            cin >> v >> u >> w;
            g[v].push_back({u, w});
        } else if (op == 2) {
            int v, l, r, w;
            cin >> v >> l >> r >> w;
            change(root1, 1, n, l, r, v, w, 0);
        } else {
            int v, l, r, w;
            cin >> v >> l >> r >> w;
            change(root2, 1, n, l, r, v, w, 1);
        }
    }
    for (int i = 0; i < 10 * N; i++) dis[i] = INF;
    priority_queue<PLI, vector<PLI>, greater<PLI>> heap;
    dis[s] = 0;
    heap.push({0, s});
    while (heap.size()) {
        int u = heap.top().second;
        heap.pop();
        if (st[u]) continue;
        st[u] = true;
        for (auto [v, w] : g[u]) {
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                heap.push({dis[v], v});
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        cout << (dis[i] == INF ? -1 : dis[i]) << " \n"[i == n];
    }

   return 0;
}
```

扫描线

1. **求矩形面积并**:

```
#include <bits/stdc++.h>

const int N = 1e4 + 10;

typedef long long LL;

struct Info {
    int minv, minvcnt;
```

```
    void apply(int t) {
        minv += t;
    }

};


struct Tag {
    int add = 0;
    void apply(int t) {
        add += t;
    }
};

Info operator+(const Info& a, const Info& b) {
    Info res;
    if (a.minv < b.minv) res = a;
    else if (b.minv < a.minv) res = b;
    else res = a, res.minvcnt += b.minvcnt;
    return res;
}


Info info[8 * N];
Tag tag[8 * N];

void pull(int u) {
    info[u] = info[u << 1] + info[u << 1 | 1];
}

void apply(int u, int t) {
    info[u].apply(t);
    tag[u].apply(t);
}

void push(int u) {
    if (tag[u].add) {
        apply(u << 1, tag[u].add);
        apply(u << 1 | 1, tag[u].add);
        tag[u] = Tag();
    }
}

std::vector<int> vy;

void build(int u, int l, int r) {
    if (l == r) {
        info[u] = {0, vy[r] - vy[r - 1]};
        return;
    }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    pull(u);
}
```

```
void modify(int u, int l, int r, int ql, int qr, int t) {
    if (l >= ql && r <= qr) {
        apply(u, t);
        return;
    }
    push(u);
    int mid = (l + r) >> 1;
    if (ql <= mid) modify(u << 1, l, mid, ql, qr, t);
    if (qr > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, t);
    pull(u);
}


int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);


    int n;
    std::cin >> n;

    std::vector<std::array<int, 4>> event;

    for (int i = 0; i < n; i++) {
        int x1, y1, x2, y2;
        std::cin >> x1 >> y1 >> x2 >> y2;
        if (x1 > x2) std::swap(x1, x2);
        if (y1 > y2) std::swap(y1, y2);
        vy.push_back(y1);
        vy.push_back(y2);
        event.push_back({x1, 1, y1, y2});
        event.push_back({x2, -1, y1, y2});
    }

    std::sort(vy.begin(), vy.end());

    vy.erase(std::unique(vy.begin(), vy.end()), vy.end());

    std::sort(event.begin(), event.end());

    int m = (int)vy.size() - 1;

    build(1, 1, m);

    int ans = 0;
    int totlen = info[1].minvcnt;
    for (int i = 0; i < event.size(); i++) {
         int l = std::lower_bound(vy.begin(), vy.end(), event[i][2]) - vy.begin()
+ 1;
         int r = std::lower_bound(vy.begin(), vy.end(), event[i][3]) -
vy.begin();
         if (i) {
            ans += (event[i][0] - event[i - 1][0]) * (totlen - (info[1].minv == 0
? info[1].minvcnt : 0));
         }
         modify(1, 1, m, l, r, event[i][1]);
```

```
    }

    std::cout << ans << '\n';

    return 0;
}
```

2. **求平面内至少被一个矩形包含的点的个数**

   这类问题通常考察求满足条件的区间个数，如果我们得出来：$l \in [x_1, x_2], r \in [y_1, y_2]$，那么方案数就为 $(x_2 - x_1 + 1)(y_2 - y_1 + 1)$，这里可以看成由 (x_1, y_1, x_2, y_2) 构成的矩形内整点的个数。

   同样也是类似维护矩形面积并，但要注意事件的顺序，且要特别注意端点不要算重，这里不是算面积了。

   最好是按时间轴一列列扫过去，这样相当于把矩形缩成线段考虑，然后其它维护手法一致。

```
#include <bits/stdc++.h>

typedef long long LL;

const int N = 2e5 + 10;

int a[N], pre[N], nxt[N], cnt[N];

struct Info {
    int miv, mivcnt;
};

Info operator+(const Info& a, const Info& b) {
    Info res;
    if (a.miv < b.miv) res = a;
    else if (a.miv > b.miv) res = b;
    else res.miv = a.miv, res.mivcnt = a.mivcnt + b.mivcnt;
    return res;
}

Info info[8 * N];

int tag[8 * N];

void pull(int u) {
    info[u] = info[u << 1] + info[u << 1 | 1];
}

void apply(int u, int t) {
    tag[u] += t;
    info[u].miv += t;
}

void push(int u) {
    if (tag[u]) {
        apply(u << 1, tag[u]);
        apply(u << 1 | 1, tag[u]);
        tag[u] = 0;
```

```
    }
}

void build(int u, int l, int r) {
    if (l == r) {
        info[u] = {0, 1};
        return;
    }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    pull(u);
}

void modify(int u, int l, int r, int ql, int qr, int x) {
    if (l >= ql && r <= qr) {
        apply(u, x);
        return;
    }
    push(u);
    int mid = (l + r) >> 1;
    if (ql <= mid) modify(u << 1, l, mid, ql, qr, x);
    if (qr > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, x);
    pull(u);
}

std::vector<std::array<int, 3>> seg[N];

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int n;

    std::cin >> n;

    for (int i = 1; i <= n; i++) {
        std::cin >> a[i];
    }

    for (int i = 1; i <= n; i++) {
        if (cnt[a[i]]) pre[i] = cnt[a[i]] + 1;
        else pre[i] = 1;
        cnt[a[i]] = i;
    }
    for (int i = 1; i <= n; i++) {
        cnt[i] = 0;
    }
    for (int i = n; i >= 1; i--) {
        if (cnt[a[i]]) nxt[i] = cnt[a[i]] - 1;
        else nxt[i] = n;
        cnt[a[i]] = i;
    }

    std::vector<std::array<int, 4>> event;
```

```
    for (int i = 1; i <= n; i++) {
        int x1 = pre[i], x2 = i;
        int y1 = i, y2 = nxt[i];
        seg[y1].push_back({x1, x2, 1});
        seg[y2 + 1].push_back({x1, x2, -1});
    }

    build(1, 1, n);

    LL ans = 0;

    for (int i = 1; i <= n; i++) {
        for (auto [l, r, v] : seg[i]) {
            modify(1, 1, n, l, r, v);
        }
        ans += n - (info[1].miv == 0 ? info[1].mivcnt : 0);
    }

    std::cout << ans << '\n';

    return 0;
}
```

主席树

```
typedef long long LL;
const int N = 1e5 + 10, INF = 1e9;
struct HJT_tree {
    struct node {
        int l, r;
        int cnt;
        LL sum;
    }tr[N * 40];
    int root[N], idx;
    void insert(int &u, int v, int l, int r, int x) {
        u = ++idx;
        tr[u] = tr[v];
        tr[u].cnt++, tr[u].sum += x;
        if (l == r) return ;
        int mid = (l + r) >> 1;
        if (x <= mid) insert(tr[u].l, tr[v].l, l, mid, x);
        else insert(tr[u].r, tr[v].r, mid + 1, r, x);
    };
    //找区间[l, r] 第 k 小
    int query(int p, int q, int l, int r, int k) {
        if (l == r) return l;
        int cnt = tr[tr[q].l].cnt - tr[tr[p].l].cnt;
        int mid = (l + r) >> 1;
        if (k <= cnt) return query(tr[p].l, tr[q].l, l, mid, k);
        else  return query(tr[p].r, tr[q].r, mid + 1, r, k - cnt);
    }
    //求区间[l, r] 前k大或者前k小的和
```

```
    LL query(int u, int v, int l, int r, int k) {
        //求区间前k大的和（前k大就优先看右子树，前k小就优先看左子树）
        if (l == r) return 1LL * l * k;
        int sz = tr[tr[u].r].cnt - tr[tr[v].r].cnt;
        int mid = (l + r) >> 1;
        if (sz >= k) return query(tr[u].r, tr[v].r, mid + 1, r, k);
        else return query(tr[u].l, tr[v].l, l, mid, k - sz) + tr[tr[u].r].sum -
tr[tr[v].r].sum;
    }
};
```

莫队

对于一类没有强制在线的区间询问问题，且添加和删除一个数是比较容易的，可以考虑离线莫队算法。

莫队算法是一种利用将询问区间按某种方法排序，使得总的时间复杂度达到 $O(n\sqrt{n})$ 的算法。

排序方法：

```
const int B = 500;
std::sort(qry, qry + q, [&](std::array<int, 4> &x, std::array<int, 4> &y){
    int c = x[0] / B;
    if (c != y[0] / B) return c < y[0] / B;
    return c % 2 ? x[1] > y[1] : x[1] < y[1];
});
```

先按询问区间的左端点按块所在编号从小到大排序，然后再利用奇偶化按右端点排序。

核心操作：

```
auto add = [&](int x) {
    cnt[x]++;
    //维护具体的答案
};
auto del = [&](int x) {
    cnt[x]--;
    //维护具体的答案
};
int l = 1, r = 0;
for (int i = 0; i < q; i++) {
    //先扩区间在缩区间
    while (l > qry[i][0]) add(--l);
    while (r < q[i][1]) add(++r);
    while (l < qry[i][0]) del(l++);
    while (r > q[i][1]) del(r--);
}

//对于一些需要维护前缀和的东西，由于要 s[r] ^ s[l - 1]
//我们不放让所有询问的左端点都 -1，这样初始化为 l = 0, r = -1 即可
//这样就可以把下标统一了
```

常见的莫队典题：

- 询问区间不同数的个数
- 询问区间内出现次数恰好为 k 的数有几个。
- 询问区间满足什么性质的 a_i, a_j 的对数。

对顶堆/multiset

可以用来动态维护前 k 小，中位数的信息（支持增加和删除）。

```
long long res = 0;
void balance() {
  if (s1.size() < k && s2.size()) {
     int t = *s2.begin();
     res += t;
     s1.insert(t);
     s2.erase(s2.begin());
  }
  if (s1.size() > k) {
     int t = *s1.rbegin();
     res -= t;
     s2.insert(t);
     s1.erase(prev(s1.end());
  }
}
void add(int x) {
  if (s2.size() && x >= *s2.begin()) s2.insert(x);
  else s1.insert(x), res += x;
  balance();
}
void del(int x) {
  auto it = s1.find(x);
  if (it != s1.end()) res -= *it, s1.erase(it);
  else s2.erase(s2.find(x));
  balance();
}
```

珂朵莉树

核心思想用 set 维护连续段

```
struct ODT {
    struct node {
        int l, r;
        mutable LL v;
        node(int l, int r = -1, LL v = 0) : l(l), r(r), v(v) {}
        bool operator<(const node &o) const {
            return l < o.l;
        }
    };
    set<node> s;
    ODT() {
        s.clear();
    }
```

```
    auto split(int pos) {
        auto it = s.lower_bound(node(pos));
        if (it != s.end() && it->l == pos) return it;
        it--;
        int l = it->l, r = it->r;
        LL v = it->v;
        s.erase(it);
        s.insert(node(l, pos - 1, v));
        return s.insert(node(pos, r, v)).first;
    }
    void assign(int l, int r, LL x) {
        auto itr = split(r + 1), itl = split(l);
        s.erase(itl, itr);
        s.insert(node(l, r, x));
    }
    void add(int l, int r, LL x) {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; it++) {
            it->v += x;
        }
    }
    LL kth(int l, int r, int k) {
        vector<pair<LL, int>> a;
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; it++) {
            a.push_back(pair<LL, int>(it->v, it->r - it->l + 1));
        }
        sort(a.begin(), a.end());
        for (auto [val, len] : a) {
            k -= len;
            if (k <= 0) return val;
        }
    }
    LL power(LL a, int b, int mod) {
        a %= mod;
        LL res = 1;
        for (; b; b /= 2, a = a * a % mod) {
            if (b % 2) {
                res = res * a % mod;
            }
        }
        return res;
    }
    LL powersum(int l, int r, int x, int mod) {
        auto itr = split(r + 1), itl = split(l);
        LL ans = 0;
        for (auto it = itl; it != itr; it++) {
            ans = (ans + power(it->v, x, mod) * (it->r - it->l + 1) % mod) % mod;
        }
        return ans;
    }
};
```

二维数点问题

可以采用扫描线解决满足：

- $x_1 \le x \le x_2$，$y_1 \le y \le y_2$ 的 (x, y) 数量。

```
//一般用按第一维 x 排序，用树状数组维护第二维 y
std::vector<std::array<int, 4>> event; //事件
{第一维，事件种类，第二维，第几个询问}。
一般原序列的点设种类为0，询问的点的种类 > 0
每次扫描线发生插入，在查询

//(x1, y1) (x2, y2)
//其中2表示 + 1, 1 表示 -1
event.push_back({x2, 2, y2, i});
event.push_back({x1 - 1, 2, y1 - 1, i});
event.push_back({x2, 1, y1 - 1, i});
event.push_back({x1 - 1, 1, y2, i});
```

启发式合并

操作：

`1 x y` 把盒子 x 里面的球全移动到盒子 y 中。

`2 x` 新增一个编号为 $k+1$ 的小球到盒子 x 中，k 为当前所有盒子球的个数之和。

`3 x` 查询编号为 x 的球在哪个盒子里。

维护套路：由于启发式合并涉及交换两个集合，这里需要做映射：

- id_i：表示编号为 i 的盒子所对应的集合编号。
- di_i：表示编号为 i 的集合所对应的盒子的编号。
- pos_i：表示编号为 i 的球所对应的集合编号。

```
#include <bits/stdc++.h>


const int N = 6e5 + 10;

std::vector<int> box[N];

int pos[N], id[N], di[N];

//pos 表示编号为 x 的球对应哪个集合编号
//id 表示编号为 x 的 box 对应哪个集合编号
//di 表示集合编号为 x 的球对应哪个 box

int main(){
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int N, Q;
```

```
    std::cin >> N >> Q;

    for (int i = 1; i <= N; i++) {
        pos[i] = id[i] = di[i] = i;
        box[i].push_back(i);
    }

    int tot = N;

    while (Q--) {
        int op;
        std::cin >> op;
        if (op == 1) {
            int x, y;
            std::cin >> x >> y;
            if (box[id[x]].size() < box[id[y]].size()) {
                std::swap(id[x], id[y]);
                std::swap(di[id[x]], di[id[y]]);
            }
            for (auto& c : box[id[y]]) {
                pos[c] = id[x];
                box[id[x]].push_back(c);
            }
            box[id[y]].clear();
        } else if (op == 2) {
            int x;
            std::cin >> x;
            pos[++tot] = id[x];
            box[id[x]].push_back(tot);
        } else {
            int x;
            std::cin >> x;
            std::cout << di[pos[x]] << '\n';
        }
    }


    return 0;
}
```

树上问题

LCA

树上倍增求 LCA

预处理 $O(n\log n)$，单次查询 $O(\log)$。

```
const int N = 1e5 + 10, M = 20;
int dep[N], f[N][M + 1];
std::vector<int> g[N];
void dfs(int u, int fa) {
    dep[u] = dep[fa] + 1;
    f[u][0] = fa;
    for (int i = 1; i <= M; i++) {
        f[u][i] = f[f[u][i - 1]][i - 1];
    }
    for (auto v : g[u]) {
        if (v == fa) continue;
        dfs(v, u);
    }
}

int LCA(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    for (int i = M; i >= 0; i--) {
        if (dep[f[u][i]] >= dep[v]) {
            u = f[u][i];
        }
    }
    if (u == v) return u;
    for (int i = M; i >= 0; i--) {
        if (f[u][i] != f[v][i]) {
            u = f[u][i];
            v = f[v][i];
        }
    }
    return f[u][0];
}
```

欧拉序 + RMQ 求 LCA

```
#include <bits/stdc++.h>
typedef std::pair<int, int> PII;
const int N = 1e5 + 10, M = 20;
std::vector<int> g[N];
int dfn[N], dep[N], Log[2 * N], idx;
PII f[2 * N][M + 1];
void dfs(int u, int fa) {
    dep[u] = dep[fa] + 1;
    dfn[u] = ++idx;
    f[idx][0] = {dep[u], u};
    for (auto v : g[u]) {
        if (v == fa) continue;
        dfs(v, u);
        f[++idx][0] = {dep[u], u};
    }
}

void init(int n) {
    Log[1] = 0;
    for (int i = 2; i <= n; i++) {
        Log[i] = Log[i / 2] + 1;
```

```
    }
    for (int i = 1; i <= Log[n]; i++) {
        for (int j = 1; j + (1 << i) - 1 <= n; j++) {
            f[j][i] = std::min(f[j][i - 1], f[j + (1 << (i - 1))][i - 1]);
        }
    }
}

int LCA(int a, int b) {
    int l = dfn[a], r = dfn[b];
    if (l > r) std::swap(l, r);
    int len = Log[r - l + 1];
    return std::min(f[l][len], f[r - (1 << len) + 1][len]).second;
}
```

查询区间 $[l, r]$ 的 LCA

设 $dfn[u]$ 表示结点 u 的 dfs 序，则有如下性质：

$LCA(l, l+1, \ldots, r) = LCA(x, y)$，其中 x 是 $[l, r]$ 中 dfn 最小的哪个，y 是 $[l, r]$ 中 dfn 最大的那个。

树的直径

```
int max_d = 0;
std::function<int(int, int)> dfs = [&](int u, int fa) {
  int d1 = d2 = 0;
  for (auto v : g[u]) {
    if (v == fa) continue;
    auto dv = dfs(v, u);
    if (dv + 1 > d1) {
      d2 = d1;
      d1 = dv + 1;
    } else if (dv + 1 > d2) {
      d2 = dv + 1;
    }
  }
  max_d = std::max(max_d, d1 + d2);
};
```

树的重心

若删除树上的某点 u 后，得到若干棵子树，使得最大子树最小，则称这个点为重心。

重心最多两个，且此时两个重心相邻

```
int root = 0, max_sz = n + 1;
std::vector<int> sz(n + 1);
auto get = [&](auto self, int u, int fa) -> void {
  sz[u] = 1;
  int res = 0;
```

```
  for (auto v : g[u]) {
    if (v == fa) continue;
    sz[u] += sz[v];
    res = std::max(res, sz[v]);
  }
  res = std::max(res, n - sz[u]);
  if (res < max_sz) {
    max_sz = res;
    root = u;
  }
};
```

树上差分

边差分

```
d = LCA(u, v)
s[u] += w;
s[v] += w;
s[d] -= 2 * w;
```

点差分

```
d = LCA(u, v)
s[u] += w;
s[v] += w;
s[d] -= w;
s[f[d][0]] -= w;
```

子树差分

```
给 u 子树加 x
s[u] += x

最后儿子累加父亲的值即可

void dfs(int u, int fa) {
    for (auto v : g[u]) {
        if (v == fa) continue;
        f[v] += f[u];
        dfs(v, u);
    }
}
```

树哈希

树的哈希函数:

$$ha[u] = 1 + \sum_{v \in son_u} f(ha[v])$$

其中 f 为哈希函数:

```
哈希方式1:
LL h(LL x) {
    return x * x * x * 1237123 + 19260817;
}

LL f(LL x) {
    return h(x & ((1LL << 31) - 1)) + h(x >> 31);
}

--------------------------------------------------------------------------------
哈希方式2:

const u64 mask = chrono::steady_clock::now().time_since_epoch().count();

u64 shift(u64 x) {
    x ^= mask;
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    x ^= mask;
    return x;
}
```

1. 对于有根树的同构判断，只要以 $root$ 跑一次树哈希，看看 $ha[root]$ 是不是相等即可。
2. 对于无根树的同构，可以考虑求出重心（至多两个），然后以重心为根跑树哈希，
   如果有两个重心就看看 $\min\{ha[r_1], ha[r_2]\}$ 是不是一样，否则直接看 $ha[r]$ 是不是相等即可。

板子:

```
LL h(LL x) {
    return x * x * x * 1237123 + 19260817;
}

LL f(LL x) {
    return h(x & ((1LL << 31) - 1)) + h(x >> 31);
}

auto get_treehash = [&](int m, std::vector<std::vector<int>> &g) -> LL {
        //求树的重心
        int ms = m + 1;
        function<void(int, int)> dfs = [&](int u, int fa) {
            sz[u] = 1, maxs[u] = 0;
            for (auto v : g[u]) {
                if (v == fa) continue;
                dfs(v, u);
                sz[u] += sz[v];
                maxs[u] = max(maxs[u], sz[v]);
            }
            maxs[u] = max(maxs[u], m - sz[u]);
```

```
            ms = min(ms, maxs[u]);
        };
        dfs(1, 0);


        function<void(int, int)> get_hash = [&](int u, int fa) {
            ha[u] = 1;
            for (auto v : g[u]) {
                if (v == fa) continue;
                get_hash(v, u);
                ha[u] += f(ha[v]);
            }
        };

        LL res = -1;

        for (int i = 1; i <= m; i++) {
            if (maxs[i] == ms) {
                get_hash(i, 0);
                if (res == -1) res = ha[i];
                else res = min(res, ha[i]);
            }
        }
        return res;
    };
```

树上启发式合并

```
int l[N], r[N], id[N], sz[N], ans[N], hs[N], tot;

//预处理子树大小，dfs序，重儿子
void dfs_init(int u, int fa) {
    l[u] = ++tot;
    id[tot] = u;
    sz[u] = 1, hs[u] = -1;
    for (auto v : g[u]) {
        if (v == fa) continue;
        dfs_init(v, u);
        sz[u] += sz[v];
        if (hs[u] == -1 || sz[v] > sz[hs[u]]) {
            hs[u] = v;
        }
    }
    r[u] = tot;
}

void dfs(int u, int fa, bool keep) {
    //keep表示是否保留此节点子树的信息
    //先处理轻儿子
    for (auto v : g[u]) {
        if (v != fa && v != hs[u]) {
            dfs(v, u, false);
        }
```

```
    }
    //处理重儿子
    if (~hs[u]) {
        dfs(hs[u], u, true);
    }
    //合并所有轻儿子信息（这里通常用作查询一些信息，例如路径）
    for (auto v : g[u]) {
        if (v == fa || v == hs[u]) continue;
        for (int i = l[v]; i <= r[v]; i++) {
            add(id[i]);
        }
    }
    //加入u本身
    add(u);
    //计算一些子树的答案
    //.....
    if (!keep) {
        //清空信息
        for (int i = l[u]; i <= r[u]; i++) {
            del(id[i]);
        }
    }
}
```

点分治

```
typedef long long LL;

const int N = 1e5 + 10;
std::vector<array<int, 2>> g[N];
int sz[N], maxs[N];
bool del[N];
LL ans, L;

void solve(int u, int s) {
    int ms = s + 1, root = -1;
    std::function<void(int, int)> self = [&](int u, int par) {
        sz[u] = 1;
        maxs[u] = 0;
        for (auto [v, w] : g[u]) {
            if (del[v] || v == par) continue;
            self(v, u);
            sz[u] += sz[v];
            maxs[u] = std::max(maxs[u], sz[v]);
        }
        maxs[u] = std::max(maxs[u], s - sz[u]);
        if (maxs[u] < ms) ms = std::maxs[u], root = u;
    };
    self(u, -1);
    //求经过root点的答案
    std::vector<int> d1, d2;
    d1.push_back(0);
    auto calc = [&](std::vector<int> &d) {
        std::sort(d.begin(), d.end());
        int m = d.size();
```

```
        int j = m - 1;
        LL res = 0;
        for (int i = 0; i < m; i++) {
            while (j >= 0 && d[j] + d[i] > L) j--;
            if (i < j) res += j - i;
        }
        return res;
    };
    for (auto [v, w] : g[root]) {
        if (del[v]) continue;
        d2.clear();
        std::function<void(int, int, int)> dfs = [&](int u, int par, int dep) {
            d1.push_back(dep);
            d2.push_back(dep);
            sz[u] = 1;
            for (auto [v, w] : g[u]) {
                if (del[v] || v == par) continue;
                dfs(v, u, dep + w);
                sz[u] += sz[v];
            }
        };
        dfs(v, u, w);
        ans -= calc(d2);
    }
    ans += calc(d1);
    del[root] = true;
    //分治下去，求不经过root点的答案
    for (auto [v, w] : g[root]) {
        if (!del[v]) solve(v, sz[v]);
    }
}
```

树链剖分

```
树链剖分的两次dfs:

int fa[N], dfn[N], top[N], dep[N], id[N], hs[N], sz[N], tot;
void dfs1(int u, int f) {
    dep[u] = dep[f] + 1;
    fa[u] = f;
    sz[u] = 1;
    hs[u] = -1;
    for (auto v : g[u]) {
        if (v == f) continue;
        dfs1(v, u);
        sz[u] += sz[v];
        if (hs[u] == -1 || sz[hs[u]] < sz[v]) {
            hs[u] = v;
        }
    }
```

```
}

void dfs2(int u, int t) {
    dfn[u] = ++tot;
    id[tot] = u;
    top[u] = t;
    if (hs[u] != -1) {
        dfs2(hs[u], t);
    }
    for(auto v : g[u]) {
        if (v == fa[u] || v == hs[u]) continue;
        dfs2(v, v);
    }
}
int lca(int u, int v) {
    while (top[u] != top[v]) {
        if (dep[top[u]] > dep[top[v]]) u = fa[top[u]];
        else v = fa[top[v]];
    }
    return dep[u] <= dep[v] ? u : v;
}
-------------------------------------------------------------------------------
-----
线段树维护重链信息部分：

struct node {
    int l, r;
    int maxv, sum;
}tr[4 * N];

void pushup(int u) {
    tr[u].maxv = max(tr[u << 1].maxv, tr[u << 1 | 1].maxv);
    tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
}
void pushup(node &x, node y) {
    x.maxv = max(x.maxv, y.maxv);
    x.sum += y.sum;
}

void build(int u, int l, int r) {
    if (l == r) {
        tr[u] = {l, r, w[id[l]], w[id[l]]};
        return ;
    }
    tr[u] = {l, r};
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    pushup(u);
}

void modify(int u, int x, int d) {
    if (tr[u].l == x && tr[u].r == x) {
        tr[u].maxv = tr[u].sum = d;
        return ;
    }
```

```
    int mid = (tr[u].l + tr[u].r) >> 1;
    if (x <= mid) modify(u << 1, x, d);
    else modify(u << 1 | 1, x, d);
    pushup(u);
}

node query(int u, int l, int r) {
    if (tr[u].l >= l && tr[u].r <= r) {
        return {0, 0, tr[u].maxv, tr[u].sum};
    }
    node res;
    res.maxv = -1e9, res.sum = 0;
    int mid = (tr[u].l + tr[u].r) >> 1;
    if (l <= mid) pushup(res, query(u << 1, l, r));
    if (r > mid) pushup(res, query(u << 1 | 1, l, r));
    return res;
}


node ask(int u, int v) {
    node res;
    res.maxv = -1e9, res.sum = 0;
    while (top[u] != top[v]) {
        if (dep[top[u]] > dep[top[v]]) {
            pushup(res, query(1, dfn[top[u]], dfn[u]));
            u = fa[top[u]];
        } else {
            pushup(res, query(1, dfn[top[v]], dfn[v]));
            v = fa[top[v]];
        }
    }
    if (dep[u] < dep[v]) {
        pushup(res, query(1, dfn[u], dfn[v]));
    } else {
        pushup(res, query(1, dfn[v], dfn[u]));
    }
    return res;
}
```

虚树

用来解决多次询问关键点，抽离查询点和其两两的 LCA 以及树根，保持树的原先结构，只不过把树的点缩小了，虚数的点数大概是 $2k$ 级别的，其中 k 是询问点。

```
//预处理dfn，以及求LCA(倍增，或者树链剖分)

void build_virtual_tree(std::vector<int> a) {
    int n = a.size();
    std::sort(a.begin(), a.end(), [](int x, int y) {
```

```
        return dfn[x] < dfn[y];
    });
    for (int i = 0; i + 1 < n; i++) {
        a.push_back(LCA(a[i], a[i + 1]));
    }
    a.push_back(1);
    std::sort(a.begin(), a.end(), [](int x, int y) {
        return dfn[x] < dfn[y];
    });
    a.erase(std::unique(a.begin(), a.end()), a.end());
    for (int i = 0; i + 1 < a.size(); i++) {
        H[LCA(a[i], a[i + 1])].push_back(a[i + 1]);
    }
}

void clear(int u) {
    for (auto v : H[u]) clear(v);
    H[u].clear();
}
```

Kruskal 重构树

常用来解决，图中任意两个点所有可行路径中的最小值最大，或者最大值最小，一些关于路径边权的问题。

考虑 Kruskal 的过程，贪心的按边权从小到大，或者从大到小排序，建立生成树，然后在建立的过程中。

对于一条边 $u-v$ 边权为 w，则新建一个点 p，让 p 的左右儿子分别等于 u,v，且 p 的点权为该边的边权 w。

性质：

- 新建的节点的点权从根节点往下越来越小（如果按边权从小到大排序）。
- 查询任意两个点的可行路径最小值最大，或者最大值最小 $\iff$ 求任意两个结点在重构树中的 LCA 的点权。

```
int p[N], n;
std::vector<int> g[N];
int val[N];
void build_Kruskal_tree(int n, int m, std::vector<edge>& e) {
    std::sort(e.begin(), e.end(), [](edge& x, edge& y) {
        return x.w > y.w;
    });
    for (int i = 1; i <= 2 * n; i++) {
        p[i] = i;
    }
    for (int i = 0; i < m; i++) {
        int u = e[i].u, v = e[i].v, w = e[i].w;
        u = find(u), v = find(v);
        if (u == v) continue;
        n++;
        p[u] = p[v] = n;
```

```
        g[n].push_back(u);
        g[n].push_back(v);
        val[n] = w;
    }
}
```

分块

```
int st[N], ed[N], bel[N], sz[N]; //块的左端点，右端点，以及每个点属于第几个块，第i块的大小
int sq = sqrt(n);
for (int i = 1; i <= sq; i++) {
    st[i] = n / sq * (i - 1) + 1;
    ed[i] = n / sq * i;
}
ed[sq] = n;
for (int i = 1; i <= sq; i++) {
    for (int j = st[i]; j <= ed[i]; j++) {
        bel[j] = i;
    }
    sz[i] = ed[i] - st[i] + 1;
}
```

分块常见维护套路：

- 如果在修改和查询在块内，直接暴力即可。
- 否则考虑对于零散的块暴力，中间跨过的块采用打标记等方法维护。
- 对于查询某个区间 $\geq$ 某个数，可以采取对于每一个块维护一个 vector ，然后暴力排序。这样每次复杂度是 $\sqrt{n} \times \log \sqrt{n}$。

Trie 树

字典树经常用来解决前缀，计算多个串的贡献的问题，01 字典树也常常用来解决异或等问题。

```
const int N =1e5 + 10, M = 26;
struct Trie {
    //注意字符串的Trie，N：一般要和总点数有关（一般就是题目输入的字符串长度之和）
    int tr[N][M], cnt[N], ed[N], idx = 0;
    void insert(string &s) {
        int p = 0;
        for (auto c : s) {
            int u = c - 'a';
            if (!tr[p][u]) tr[p][u] = ++idx;
            p = tr[p][u];
            cnt[p]++;
        }
        ed[p]++;
    }
    bool query(string &s) {
        int p = 0;
        for (auto c : s) {
            int u = c - 'a';
```

```
            p = tr[p][u];
            if (!p) return false;
        }
        return true;
    }
};


struct 01Trie {
    int tr[N * M][2], cnt[N * M], ed[N * M], idx = 0;
    void insert(int x) {
        int p = 0;
        for (int i = 30; i >= 0; i--) {
            int u = x >> i & 1;
            if (!tr[p][u]) tr[p][u] = ++idx;
            p = tr[p][u];
            cnt[p]++;
        }
        ed[p]++;
    }
    //查询和 x 异或最大
    int query(int x) {
        int p = 0, res = 0;
        for (int i = 30; i >= 0; i--) {
            int u = x >> i & 1;
            if (tr[p][!u]) p = tr[p][!u], res += 1 << i;
            else p = tr[p][u];
        }
        return res;
    }
};

//可持久化01树
struct long_01Trie {
    int tr[N * M][2], id[N * M], root[N * M], idx = 0;
    void insert(int u, int v, int x, int type) {
        for (int i = 30; i >= 0; i--) {
            int c = x >> i & 1;
            tr[u][!c] = tr[v][!c];
            tr[u][c] = ++idx;
            u = tr[u][c], v = tr[v][c];
            id[u] = type;
        }
    }
    //查询和 x 异或最大
    int query(int u, int L, int x) {
        int res = 0;
        for (int i = 30; i >= 0; i--) {
            int c = x >> i & 1;
            if (id[tr[u][!c]] >= L) u = tr[u][!c], res += 1 << i;
            else u = tr[u][c];
        }
        return res;
    }
};
//初始化
```

```
id[0] = -1, root[0] = 0;
//每次插入:
root[i] = ++idx;
insert(root[i], root[i - 1], x, i);
```

字符串

KMP 和 Border

求 nxt 数组和 KMP 板子

```
nxt[0] = nxt[1] = 0;
for (int i = 2, j = 0; i <= n; i++) {
    while (j && s[i] != s[j + 1]) j = nxt[j];
    if (s[i] == s[j + 1]) j++;
    nxt[i] = j;
}
```

```
for (int i = 1, j = 0; i <= n; i++) {
    while (j && s[i] != t[j + 1]) j = nxt[j];
    if (s[i] == t[j + 1]) j++;
    if (j == m) {
        ...
        j = nxt[j];
    }
}
```

一个字符串 S 的所有 Border 构成了一个border树，有 $n+1$ 个节点：$0,1,2,\ldots,n$，每个节点 i 的父亲为 $next[i]$。

Border 的性质：

- 如果 p 是字符串的一个周期，那么 $|s|-p$ 是字符串的一个 Border (我们这里称 Border 是长度)。
- 求字符串所有 Border $\iff$ 求字符串所有前缀的最大 Border。只要一直跳 nxt 数组即可：

  ```
  int p = nxt[n];
  while (p) p = nxt[p];
  ```

- 一个串的 border 数量是 $O(n)$ 级别的，它们组成了 $\log(n)$ 个等差数列。
- 若 p,q 都是字符串的周期，那么 $\gcd(p,q)$ 也是字符串的周期。

Border 树的性质：

- 建树方式：$i \to nxt[i]$ 连边。
- 每个前缀 $pre[i]$ 的所有 Border 为节点 i 到根的链。
- 有哪些前缀有长度为 x 的 Border，相当于求 x 的子树大小。
- 求两个前缀的公共 Border 等价于求两个节点的 LCA。

马拉车

```
std::vector<int> Manacher(std::string &a) {
    std::string s = "?#";
    for (auto &c : a) {
        s += c;
        s += '#';
    }
    int len = s.size() - 1;
    std::vector<int> d(len + 1);
    int maxr = 0, mid = 1;
    for (int i = 1; i <= len; i++) {
        d[i] = (i < maxr ? std::min(d[2 * mid - i], maxr - i + 1) : 1);
        while (s[i - d[i]] == s[i + d[i]]) d[i]++;
        if (i + d[i] - 1 > maxr) {
            maxr = i + d[i] - 1;
            mid = i;
        }
    }
    a = s;
    return d;
}
```

1. d_i 表示以 i 为中心的最大回文半径，即最大的 d_i 满足 $[i-d_i+1, i+d_i-1]$ 是回文串。
2. 原串长度为 n，加入 `'#'` 后，新串长度为 $2n+1$，若 `s[i]= '#'`，则表示其为偶回文串的中心，反之为奇回文串中心。
3. 在新串中下标为 i 的位置对应原串下标为 $\frac{i}{2}$。
4. 以 i 为中心的最大回文串长度为 d_i-1。
5. 计算所有回文子串个数 `ans += d[i] / 2`。
6. 本质不同的回文串数量是 $O(n)$ 的。

题目：

[【模板】manacher]

[SHOI2011双倍回文]（利用本质不同回文串只有 n 的性质）

[[国家集训队] 最长双回文串]（枚举拼接点 `#`，维护 l_i，r_i 以 i 为左端点和右端点最长回文串长度，利用递推）

[这是回文吗]（也是处理 l_i，r_i 以 i 为左端点和右端点最长回文串长度）

`l[i] = std::max(l[i], l[i - 1] - 2)`

`r[i] = std::max(r[i], r[i + 1] - 2)`

字符串哈希

支持多维度哈希

```
// 支持多维度哈希，可自定义修改
#define DIMS 2
#define HASH_ARRAY std::array<int, DIMS>
```

```
const int MOD_VEC[] = {1000000007, 1000000009, 1000000033, 1000000037,
1000000041};
const int BASE_VEC[] = {127, 1331, 131, 353, 1543};
std::mt19937
rng(std::chrono::high_resolution_clock::now().time_since_epoch().count());
// 全局随机初始化 Base, Mod
HASH_ARRAY Base = []() {
  HASH_ARRAY arr;
  int base_size = sizeof(BASE_VEC) / sizeof(BASE_VEC[0]);
    assert(DIMS <= base_size);
  std::vector<int> vec(base_size);
  std::iota(vec.begin(), vec.end(), 0);
  std::shuffle(vec.begin(), vec.end(), rng);
  for (int i = 0; i < DIMS; i++) {
    arr[i] = BASE_VEC[vec[i]];
  }
  return arr;
}();
HASH_ARRAY Mod = []() {
  HASH_ARRAY arr;
  int mod_size = sizeof(MOD_VEC) / sizeof(MOD_VEC[0]);
    assert(DIMS <= mod_size);
  std::vector<int> vec(mod_size);
  std::iota(vec.begin(), vec.end(), 0);
  std::shuffle(vec.begin(), vec.end(), rng);
  for (int i = 0; i < DIMS; i++) {
    arr[i] = MOD_VEC[vec[i]];
  }
  return arr;
}();
// 字符串哈希模版
struct StringHash {
  std::vector<HASH_ARRAY> p, b;
  StringHash(const std::string& s) {
    // 传入的字符串s下标从0开始
    int n = s.size();
    assert(n >= 1);
    p.resize(n + 1), b.resize(n + 1);
    for (int i = 0; i < DIMS; i++) {
      p[0][i] = 0, b[0][i] = 1;
    }
    for (int i = 1; i <= n; i++) {
      for (int j = 0; j < DIMS; j++) {
                b[i][j] = 1LL * b[i - 1][j] * Base[j] % Mod[j];
        p[i][j] = (1LL * p[i - 1][j] * Base[j] + s[i - 1]) % Mod[j];
      }
    }
  }
  // 获取[l, r]哈希值
  HASH_ARRAY GetHashVal(int l, int r) {
    HASH_ARRAY val{};
    if (l > r) {
      return val;
    }
    for (int i = 0; i < DIMS; i++) {
```

```
      val[i] = (p[r][i] - 1LL * p[l - 1][i] * b[r - l + 1][i] % Mod[i] + Mod[i])
% Mod[i];
    }
    return val;
  }
  // 判断[l1, r1]与[l2, r2]哈希是否相等
  bool Equal(int l1, int r1, int l2, int r2) {
    return GetHashVal(l1, r1) == GetHashVal(l2, r2);
  }
};


// 使用例子
/*
std::string s;
std::cin >> s;
StringHash strhash(s);
*/
```

Z 函数

求 LCP（最长公共前缀）

设字符串 S，令 z_i 表示字符串 $S[i \backsim n]$ 和 $S[1 \backsim n]$ 的最长公共前缀长度。

```
z[1] = n;
for (int i = 2, l = 1, r = 1; i <= n; i++) {
    z[i] = i <= r ? std::min(z[i - l + 1], r - i + 1) : 0;
    while (i + z[i] <= n && s[i + z[i]] == s[1 + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) r = i + z[i] - 1, l = i;
}
```

AC自动机

几个核心：

$p \to go[i]$ 表示节点 p 如果下一个要匹配 i 字母，需要沿跳到哪个节点。

$p \to fail$：表示节点 p 的 $fail$，$fail$ 表示节点 p 表示的字符串的最长真后缀（真后缀指在字典树上出现过）

$go, fail$ 的求法：

核心是利用 BFS，先求出浅层节点的 $go, fail$。

$$p \to go[i] = \begin{cases} p \to son[i] & p \to son[i] \neq nullptr \\ p \to fail \to go[i] & other \end{cases}$$

$$p \to son[i] \to fail = p \to fail \to go[i]$$

```cpp
namespace acm {
    const int N = 2e5 + 10, M = 26;
    struct node {
        node *son[M], *go[M], *fail;
        int cnt;
    } pool[N], *cur = pool, *root, *d[N];

    node *newnode() {
        return cur++;
    }

    int t;
    node *q[N];

    void init() {
        cur = pool, t = 0;
        root = newnode();
    }

    void build() {
        q[t++] = root;
        for (int i = 0; i < t; i++) {
            node *u = q[i];
            for (int j = 0; j < M; j++) {
                if (u->son[j]) {
                    u->go[j] = u->son[j];
                    if (u == root) u->son[j]->fail = root;
                    else u->son[j]->fail = u->fail->go[j];
                    q[t++] = u->son[j];
                } else {
                    if (u == root) u->go[j] = root;
                    else u->go[j] = u->fail->go[j];
                }
            }
        }
    }

    void insert(const std::string &s, int id) {
        node *p = root;
        for (auto &c : s) {
            int w = c - 'a';
            if (!p->son[w]) {
                p->son[w] = newnode();
            }
            p = p->son[w];
        }
        d[id] = p;
    }
}
```

图论

最短路

迪杰斯特拉

```
int Dijstla(int n, int beg, int end, std::vector<PII> g[]) {
    std::vector<bool> st(n + 1, false);
    std::vector<int> d(n + 1, 1e9);
    std::priority_queue<PII, std::vector<PII>, std::greater<PII>> heap;
    d[beg] = 0, heap.push({0, beg});
    while (heap.size()) {
        int u = heap.top().second;
        heap.pop();
        if (st[u]) continue;
        st[u] = true;
        for (auto [v, w] : g[u]) {
            if (d[v] > d[u] + w) {
                d[v] = d[u] + w;
                heap.push({d[v], v});
            }
        }
    }
    return d[end] == 1e9 ? -1 : d[end];
}
```

SPFA

```
int spfa(int n, int beg, int end, std::vector<PII> g[]) {
    std::vector<bool> st(n + 1, false);
    std::vector<int> d(n + 1, 1e9);
    std::vector<int> cnt(n + 1, 0);
    std::queue<int> q;
    d[beg] = 0, st[beg] = true, q.push(beg);
    while (q.size()) {
        int u = q.front();
        q.pop(), st[u] = false;
        for (auto [v, w] : g[u]) {
            if (d[v] > d[u] + w) {
                d[v] = d[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= n) {
                    //出现负环
                    return -1;
                }
                if (!st[v]) {
                    st[v] = true, q.push(v);
                }
            }
        }
    }
    return d[end] == 1e9 ? -1 : d[end];
```

```
}
```

Bellman-ford

复杂度 $O(NM)$，可以求负权图、判负环等

```
int n, m, s;
cin >> n >> m >> s;

vector<tuple<int, int, i64>> ver(m + 1);
for (int i = 1; i <= m; ++i) {
    int x, y;
    i64 w;
    cin >> x >> y >> w;
    ver[i] = {x, y, w};
}

vector<i64> dis(n + 1, inf), chk(n + 1);
dis[s] = 0;
for (int i = 1; i <= 2 * n; ++i) { // 双倍松弛，获取负环信息
    vector<i64> backup = dis;
    for (int j = 1; j <= m; ++j) {
        auto [x, y, w] = ver[j];
        chk[y] |= (i > n && backup[x] + w < dis[y]);
        dis[y] = min(dis[y], backup[x] + w);
    }
}

for (int i = 1; i <= n; ++i) {
    if (i == s) {
        cout << 0 << " ";
    } else if (dis[i] >= inf / 2) {
        cout << "no ";
    } else if (chk[i]) {
        cout << "inf ";
    } else {
        cout << dis[i] << " ";
    }
}
```

Floyd

多源最短路

k, i, j，表示从 $i \to j$，只考虑中间经过 $1 \backsim k$ 的点的路径最小值

```
void floyd() {
    for (int k = 1; k <= n; k ++)
        for (int i = 1; i <= n; i ++)
            for (int j = 1; j <= n; j ++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

二分图

二分图最大匹配数 = 最小点覆盖 = 总点数 − 最大独立集 = 总点数(未拆点前) − 最小路径覆盖

二分图最大匹配：表示在二分图中选出最多的 m 条边，使得它们任意两条边都没有公共点

最小点覆盖：表示在二分图中选出最少的 m 个点，把所有的边都覆盖(即每条边的两个端点都至少被选上一个)

最大独立集：表示集合中的任意两个点都不能有边

最小路径覆盖：

表示在一个 DAG 中用最少的互不相交的路径把所有点覆盖

做法是把每一个点拆成入点和出点，然后每一条边相当于从一个出点先入点两边，是二分图

最小路径重复覆盖：先在原图求一次传递闭包，然后等价与在新图中做一次最小路径覆盖

染色法判定二分图

```
bool dfs(int u, int c) {
    color[u] = c;
    for (auto v : g[u]) {
        if (!color[v]) {
            if (!dfs(v, 3 - c)) return false;
        } else if (color[v] == c) return false;
    }
    return true;
}
```

种类并查集判断二分图

```
while (m--) {
    int a, b;
    cin >> a >> b;
    p[find(a)] = find(b + n);
    p[find(b)] = find(a + n);
}
bool ok = true;
for (int i = 1; i <= n; i++) {
    if (find(i) == find(i + n)) {
        ok = false;
        break;
    }
}
```

匈牙利算法（求二分图最大匹配）

```
bool find(int u) {
    for (auto v : g[u]) {
        if (!st[v]) {
            st[v] = true;
            if (match[v] == 0 || find(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int res = 0;
for (int i = 1; i <= n; i++) {
    memset(st, false, sizeof st);
    if (find(i)) res++;
}
```

差分约束

- 求 x_i 的最小值，我们要构造形如 $x_i \geq x_j + c_k$ 的不等式,即连一条从 $j \to i$长度为 c_k的边，然后求一遍最长路。
- 求 x_i 的最大值，我们要构造形如 $x_i \leq x_j + c_k$ 的不等式,即连一条从 $j \to i$ 长度为 c_k 的边，然后求一遍最短路。

欧拉路径/欧拉回路

欧拉路径：一笔画完图中全部边。

欧拉回路：当起点和终点相同时称欧拉回路。

有向图欧拉路径存在判定

有向图欧拉路径存在：[1] 恰有一个点出度比入度多 1 （为起点）；[2] 恰有一个点入度比出度多 1 （为终点）；[3] 恰有 $N-2$ 个点入度均等于出度。如果是欧拉回路，则上方起点与终点的条件不存在，全部点均要满足最后一个条件。

```
signed main() {
    int n, m;
    cin >> n >> m;

    DSU dsu(n + 1); // 如果保证连通，则不需要 DSU
    vector<unordered_multiset<int>> ver(n + 1); // 如果对于字典序有要求，则不能使用
unordered
    vector<int> degI(n + 1), degO(n + 1);
    for (int i = 1; i <= m; i++) {
        int x, y;
        cin >> x >> y;
        ver[x].insert(y);
        degI[y]++;
        degO[x]++;
        dsu.merge(x, y); // 直接当无向图
    }
    int s = 1, t = 1, cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (degI[i] == degO[i]) {
            cnt++;
        } else if (degI[i] + 1 == degO[i]) {
            s = i;
        } else if (degI[i] == degO[i] + 1) {
            t = i;
        }
    }
    if (dsu.size(1) != n || (cnt != n - 2 && cnt != n)) {
        cout << "No\n";
    } else {
        cout << "Yes\n";
    }
}
```

无向图欧拉路径存在判定

无向图欧拉路径存在：[1] 恰有两个点度数为奇数（为起点与终点）；[2] 恰有 $N-2$ 个点度数为偶数。

```
signed main() {
    int n, m;
    cin >> n >> m;

    DSU dsu(n + 1); // 如果保证连通，则不需要 DSU
    vector<unordered_multiset<int>> ver(n + 1); // 如果对于字典序有要求，则不能使用
unordered
    vector<int> deg(n + 1);
    for (int i = 1; i <= m; i++) {
        int x, y;
```

```
        cin >> x >> y;
        ver[x].insert(y);
        ver[y].insert(x);
        deg[y]++;
        deg[x]++;
        dsu.merge(x, y); // 直接当无向图
    }
    int s = -1, t = -1, cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (deg[i] % 2 == 0) {
            cnt++;
        } else if (s == -1) {
            s = i;
        } else {
            t = i;
        }
    }
    if (dsu.size(1) != n || (cnt != n - 2 && cnt != n)) {
        cout << "No\n";
    } else {
        cout << "Yes\n";
    }
}
```

有向图欧拉路径求解（字典序最小）

```
vector<int> ans;
auto dfs = [&](auto self, int x) -> void {
    while (ver[x].size()) {
        int net = *ver[x].begin();
        ver[x].erase(ver[x].begin());
        self(self, net);
    }
    ans.push_back(x);
};
dfs(dfs, s);
reverse(ans.begin(), ans.end());
for (auto it : ans) {
    cout << it << " ";
}
```

无向图欧拉路径求解

```
auto dfs = [&](auto self, int x) -> void {
    while (ver[x].size()) {
        int net = *ver[x].begin();
        ver[x].erase(ver[x].find(net));
        ver[net].erase(ver[net].find(x));
        cout << x << " " << net << endl;
        self(self, net);
    }
};
dfs(dfs, s);
```

Tarjan

强连通分量

```
const int N = 1e5 + 10;
int dfn[N], id[N], low[N], cnt, idx;
bool instk[N];
vector<int> scc[N], g[N];
stack<int> stk;

void tarjan(int u) {
    dfn[u] = low[u] = ++idx;
    instk[u] = true;
    stk.push(u);
    for (auto v : g[u]) {
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (instk[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] == low[u]) {
        ++cnt;
        while (true) {
            int v = stk.top();
            instk[v] = false;
            stk.pop();
            id[v] = cnt;
            scc[cnt].push_back(v);
            if (u == v) break;
        }
    }
}
```

割边

```
const int N = 1e5 + 10;
std::vector<std::array<int, 2>> g[N];
int dfn[N], low[N], pos[N], idx, edcc_cnt;
std::vector<int> bridge;
std::stack<int> stk;
std::vector<int> edcc[N];
bool is_bridge[N];
void tarjan(int u, int id) {
    dfn[u] = low[u] = ++idx;
    stk.push(u);
    for (auto [v, id2] : g[u]) {
        if (!dfn[v]) {
            tarjan(v, id2);
            low[u] = min(low[u], low[v]);
```

```
            if (low[v] > dfn[u]) bridge.push_back(id2), is_bridge[id2] = true;
        } else if(id != id2) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dnf[u] == low[u]) {
        edcc_cnt++;
        while (true) {
            int y = stk.top();
            stk.pop();
            pos[y] = edcc_cnt;
            edcc[edcc_cnt].push_back(y);
            if (y == u) break;
        }
    }
}
```

割点

```
const int N = 1e5 + 10;
std::vector<int> g[N];
int dfn[N], low[N], cut[N], id[N], idx, cnt;
std::vector<int> dcc[N];
std::stack<int> stk;
void tarjan(int u, int fa) {
    dfn[u] = low[u] = ++idx;
    int ch = 0;
    stk.push(u);
    for (auto v : g[u]) {
        if (!dfn[v]) {
            ch++;
            tarjan(v, fa);
            low[u] = min(low[u], low[v]);
            if (low[v] >= dfn[u]) {
                cut[v] = 1;
                cnt++;
                while (true) {
                    int y = stk.top();
                    stk.pop();
                    id[y] = cnt;
                    dcc[cnt].push_back(y);
                    if (y == v) break;
                }
                dcc[cnt].push_back(u);
            }
        } else if (v != fa) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (u == 1 && ch <= 1) cut[u] = 0;
}
```

scc 缩点

scc 缩点，最终会得到一个拓扑图

```
std::vector<int> NG[N];
int in[N], out[N];
void scc() {
    for (int i = 1; i <= n; i++) {
        for (auto j : g[i]) {
            if (id[i] != id[j]) {
                NG[id[i]].push_back(id[j]);
                in[id[j]]++, out[id[i]]++;
            }
        }
    }
}
```

edcc 缩点

edcc 缩点，就是把所有边双连通分量缩成一个点，最终会变成一棵树

```
std::vector<int> NG[N];

void e_dcc() {
    for (int i = 1; i <= n; i++) {
        for (auto [j, id] : g[i]) {
            if (is_bridge[id]) {
                g[pos[i]].push_back(pos[j]);
            }
        }
    }
}
```

vdcc 缩点

```
int id[N];
std::vector<int> g[N];
void v_dcc() {
    int sum = cnt;
    for (int i = 1; i <= n; i++) {
        if (cut[i]) id[i] = ++sum;
    }
    for (int i = 1; i <= cnt; i++) {
        for (auto v : dcc[i]) {
            if (cut[v]) {
                 g[id[v]].push_back(i);
                 g[i].push_back(id[v]);
            }
        }
    }
```

```
}
```

圆方树

考虑每个点双都建立一个方点，使其连向所有点双的点，那么整个图就会变成一棵树，那么任意两个方点之间的点，肯定是一个割点。

经典性质：

- 判断 $A \to C$ 的简单路径中是否经过了 B，只要看 B 是否在圆方树中 $A \to C$ 的路径上，或者在路径上的一个方点的邻点上。

```
const int N = 1e5 + 10;
std::vector<int> g[N];
int dfn[N], low[N], f[N], idx, cnt;
std::vector<int> H[N];
std::stack<int> stk;

//注意这里做之前，先让 cnt = n，因为每个方点需要重新分配一个编号给它
void tarjan(int u, int fa) {
    dfn[u] = low[u] = ++idx;
    stk.push(u);
    for (auto v : g[u]) {
        if (!dfn[v]) {
            tarjan(v, fa);
            low[u] = min(low[u], low[v]);
            if (low[v] >= dfn[u])
                cnt++;
                while (true) {
                    int y = stk.top();
                    stk.pop();
                    f[y] =cnt;
                    H[cnt].push_back(y);
                    if (y == v) break;
                }
                f[cnt] = u;
                H[u].push_back(cnt);
            }
        } else if (v != fa) {
            low[u] = min(low[u], dfn[v]);
        }
    }
}
```

基础算法

前缀和，差分

当需要快速求某一段连续区间的和的时候，就需要用到前缀和：

核心思想：

$s_1 = a_1$

$s_2 = a_1 + a_2$

$...$

$s_n = a_1 + a_2 + .. + a_n$

$s_i = s_{i-1} + a_i$

故 $\sum_{i=l}^{r} a_i = s_r - s_{l-1}$

一维前缀和：

计算前缀和 $[1, i]$ ： $s[i] = s[i-1] \ + \ a[i]$

计算区间$[l, r]$ 的和 $= s[r] - s[l-1]$

二维前缀和：

计算左上角为 $(1, 1)$，右下角为 (x, y) 的二维前缀和：
$s[x][y] = s[x-1][y] + s[x][y-1] - s[x-1][y-1] + a[x][y]$

计算左上角为 (x_1, y_1)，右下角为 (x_2, y_2) 的矩形的和 $=$
$s[x_2][y_2] - s[x_2][y_1 - 1] - s[x_1 - 1][y2] + s[x_1 - 1][y_1 - 1]$

三维前缀和：

计算左上角为 $(1, 1, 1)$ ，右下角为 (x, y, z) 的三维前缀和：

$s[x][y][z] = s[x-1][y][z] + s[x][y-1][z] + s[x][y][z-1]$

$-s[x-1][y-1][z] - s[x][y-1][z-1] - s[x-1][y][z-1] + s[x-1][y-1][z-1] + a[x][y][z]$

计算左上角为 (x_1, y_1, z_1) ，右下角为 (x_2, y_2, z_2) 的立方体的和：

$s[x_2][y_2][z_2] - s[x_1 - 1][y_2][z_2] - s[x_2][y1 - 1][z_2] - s[x_2][y_2][z_1 - 1]$

$+$
$s[x_1 - 1][y_1 - 1][z + 2] + s[x_1 - 1][y_2][z_1 - 1] + s[x_2][y_1 - 1][z_1 - 1] - s[x_1 - 1][y_1 - 1][z_1 - 1]$

多维前缀和：

我们可以根据 $1, 2, 3$ 维的前缀和总结，我们发现其本质是一个容斥原理：

我们设 n 维，我们前缀和就是要把所有的子集相加，即每一位的并集，我们可以看成 n 维求并集，就是容斥原理的应用

我们可以把 n 维数组，看成一个 n 位的 01 串，每一位都对应了选还是不选，故一共 2^n 个子集

我们的多维前缀和公式，就是可以看成选哪几维 -1 ，哪几维不变，故我们遵循容斥原理：

奇加偶减(有奇数个-1，就是加号，有偶数个-1就是减号)，

即：$s[x][y][z]..[n]=a[x][y][z]..[n]+(2^{n-1}$ 个子集的$s[x][y][z]...[n])$

若要计算 $(x_1,y_1,z_1,.....,n_1)$ 与 $(x_2,y_2,z_2,.....,n_2)$ 构成多维矩阵的和，我们同样是遵循容斥原理：

只不过我们变成了 **奇减偶加(有奇数个-1，就是减号，有偶数个-1就是加号)**：

$s[x_2][y_2][z_2]..[n_2]$ $+(2^n-1$ 个子集) 或者直接看成 2^n 个子集的 并集运算

即选几个 $x_1,y_1...n_1$ 减一，几个为 $x_2,y_2...n_2$

前缀和的前缀和：

$s[i]$ 表示 $\sum_{i=1}^{i}a[i]$

$ss[i]$ 表示 $\sum_{i=1}^{i}s[i]$

若要计算形如如下形式的和：

$a[l]+(a[l]+a[l+1])$+$(a[l]+a[l+1]+a[l+2])$+$...$+$(a[l]$ $+a[l+1]+...+a[r])$

$=ss[r]-ss[l-1]-(r-l+1)*s[l-1]$

何为差分：

我们设原数组为：$a_1,a_2,a_3,.....,a_n$

我们构造一种差分数组，使其为：

$b_1=a_1$

$b_2=a_2-a_1$

$b_3=a_3-a_2$

$....$

$b_n=a_n-a_{n-1}$

这种数组有个性质，我们发现其前缀和就是原数组，即：

$a_1=b_1$

$a_2=b_1+b_2$

$a_3=b_1+b_2+b_3$

$...$

$a_n=b_1+b_2+b_3+...+b_n$

其作用是什么，对于需要对区间 $[l,r]$ 加上某一个数 c 是，利用差分数组，我们可以把区间修改变成单点修改

我们只需要让 $b[l]+=c,b[r+1]-=c$ 即可

一维差分：

$b[l]+=c$

$b[r+1]-=c$

二维差分：

$b[x_1][y_1]+=c$

$b[x_1][y_2+1]-=c$

$b[x_2+1][y1]-=c$

$b[x_2+1][y_2+1]+=c$

三维差分：

$b[x_1][y_1][z_1]+=c$

$b[x_2+1][y_1][z_1]-=c$

$b[x_1][y_2+1][z_1]-=c$

$b[x_1][y_1][z_2+1]-=c$

$b[x_2+1][y_2+1][z_1]+=c$

$b[x_2+1][y1][z_2+1]+=c$

$b[x_1][y_2+1][z_2+1]+=c$

$b[x_2+1][y_2+1][z_2+1]-=c$

多维差分：

我们由 $1,2,3$ 维差分可以归纳总结出规律，其实还是类比容斥原理

我们相当于从 n 维选出 x 维使其：$x_2+1, y_2+1, \ldots$，剩下的 $n-x$ 都为 $x_1, y_1, \ldots$

若 x 为奇数，则为 $-=c$，否则为 $+=c$ **(奇减偶加)**

其实就是求 2^n 个子集的并集

二阶差分

令 b 为 a 的差分数组，c 为 b 的差分数组。

$b_1=a_1$

$b_2=a_2-a_1$

$\ldots$

$b_n=a_n-a_{n-1}$

$c_1=b_1$

$c_2=b_2-a_1$

$\ldots$

$c_n=b_n-b_{n-1}$

把 a 中 $[l,r]$ 加上一个首项为 x，公差为 d 的等差数列

设 $y=x+(r-l)\times d$，即末项。

$\Longleftrightarrow$

```
c[l] += x;
c[l + 1] += d - x;
c[r + 1] -= d + y;
c[r + 2] += y;
```

差分 ticks:

一般涉及区间修改，我们都可以求其差分数组，转变为单点修改问题

一些经典的差分问题：

- 每次操作选一个区间 $[l, r]$ 任意 $+1$ 或者 -1，问最少操作几次可以使得区间所有数相同

  我们可以发现所有数相同$<=>$ $b_1 = a_1, b_2 = 0, b_3 = 0, \ldots, b_n = 0$ (即 a 数组的差分数组的 $[2, n]$ 项都为 0)

  这样就区间操作，就变成了每次选两个下标的单点操作

  两个经典结论：

  设 ne 为差分数组 $[2, n]$ 项中的负数绝对值之和，pe 差分数组 $[2, n]$ 项中的正数之和

  (1) 最少操作次数：$min(pe, ne) \ + \ |ne - pe|$

  (2) 操作完成之后最终可能有几种不同的序列：$|ne - pe| \ + \ 1$

- 给你 n 个区间 $[l, r]$ 让你求出这 n 个区间的并集，然后每次询问某个数在不在区间的并集当中

  这也是一个经典的差分问题，我们每有一个区间就让 $b[l] + +, b[r + 1] - -$，这样最后求前缀和得到原数组，原数组只要 $a[i] > 0$ 则表示出现过在并集中

  否则就是没出现过，如果区间的端点范围很大，可以考虑离散化之后在差分

位运算

1. 枚举 x 的子集（相当于枚举 x 的哪些 1 变成 0）。

   ```
   for (int i = x; ; i = (i - 1) & x) {

       if (i == 0) break;
   }
   ```

   枚举 $0 \backsim 2^n - 1$ 的所有数的子集的复杂度是 $O(3^n)$。

   ```
   for (int i = 0; i < 1 << n; i++) {
       for (int j = i; j; j = (j - 1) & i) {

           if (j == 0) break;
       }
   }
   ```

2. 枚举 x 的超集（相当于枚举 x 的哪些 0 变成 1）。

   ```
   for (int i = x; i < 1 << n; i = (i + 1) | x) {

   }
   ```

枚举 $0 \backsim 2^n - 1$ 的所有数的超集的复杂度是 $O(3^n)$。

```
for (int i = 0; i < 1 << n; i++) {
    for (int j = i; j < 1 << n; j = (j + 1) | i) {

    }
}
```

3. 高维前缀和

其思想为一维一维的加上去，例如要求二维前缀和：$g[i][j]$，先对行做一次前缀和：$g[i][j] = g[i][j-1] + f[i][j]$。

在对列做：$g[i][j] = g[i][j] + g[i-1][j]$。

即对于高维数组：$g[i_1][i_2][i_3][i_4]\ldots[i_n]$，只要按 $g[][][]\ldots[i_n - 1], g[][][]..[i_{n-1} - 1][]\ldots$ 求和即可。

对于所有的 $0 \backsim 2^n - 1$，都求出 $\sum_{j \subset i} a[j]$，即 i 的所有子集和，复杂度为 $O(n2^n)$。

$i \,\&\, j = j$：表示 j 是 i 的子集。

$i \,\&\, j = 0$：表示 $\backsim i \,\&\, j = j$，即 j 是 $\backsim i$ 的子集。

```
//子集和
for (int j = 0; j < n; j++) {
    for (int i = 0; i < 1 << n; i++) {
        if (i >> j & 1) {
            cnt[i] += cnt[i ^ (1 << j)];
        }
    }
}
//超集和
for (int j = 0; j < n; j++) {
    for (int i = 0; i < 1 << n; i++) {
        if (!(i >> j & 1)) {
            cnt[i] += cnt[i | (1 << j)];
        }
    }
}
```

计算几何

```
namespace Geometry
{
    const double pi = acos(-1);
    const double eps = 1e-8;
    // 点与向量
    struct Point
    {
        double x, y;
        Point(double x = 0, double y = 0) : x(x), y(y) {}
        bool operator==(const Point a) const
        {
            return (fabs(x - a.x) <= eps && fabs(y - a.y) <= eps);
        }
    };
```

```
typedef Point Vector;
Vector operator+(Vector A, Vector B)
{
    return Vector(A.x + B.x, A.y + B.y);
}
Vector operator-(Vector A, Vector B)
{
    return Vector(A.x - B.x, A.y - B.y);
}
Vector operator*(Vector A, double p)
{
    return Vector(A.x * p, A.y * p);
}
Vector operator/(Vector A, double p)
{
    return Vector(A.x / p, A.y / p);
}

int sign(double x)
{ // 符号函数
    if (fabs(x) < eps)
        return 0;
    if (x < 0)
        return -1;
    return 1;
}
int cmp(double x, double y)
{ // 比较函数
    if (fabs(x - y) < eps)
        return 0;
    if (x < y)
        return -1;
    return 1;
}

double dot(Point a, Point b)
{   // 向量点积
    // |a| * |b| * cos(x)
    //dot可以判断两向量形成的夹角，> 0 锐角，= 0 直角，< 0 钝角
    return a.x * b.x + a.y * b.y;
}

double cross(Point a, Point b)
{   // 向量叉积
    // |a| * |b| * sin(x)
    //注意 a * b 和 b * a 是不一样的
    //x表示a逆时针转多少度到b

    //cross 可以用来判断两个向量的位置关系
    //cross(a, b) > 0 : 表示 b 在 a 的逆时针方向上
    //cross(a, b) = 0 : 表示 a 和 b 共线
    //cross(a, b) < 0 : 表示 b 在 a 的顺时针方向上

    return a.x * b.y - b.x * a.y;
}
```

```
    double get_length(Point a)
    { // 求向量模长
        return sqrt(dot(a, a));
    }
    //求向量a的方向单位向量（单位向量，模长为1或者0）
    Point unit(Point a) {
        return a / get_length(a);
    }

    double get_angle(Point a, Point b)
    { // 求A->B的有向角
        return acos(dot(a, b) / get_length(a) / get_length(b));
    }

    double area(Point a, Point b, Point c)
    { // A为顶点，向量AB与向量AC的叉积，即三角形ABC的面积的2倍（有向）
        return cross(b - a, c - a);
    }

    Point rotate_90(Point a) {
        //把点a，逆时针旋转90度
        return Point(-a.y, a.x);
    }

    Point rotate_s(Point a, double angle)
    { // 将向量A顺时针旋转angle度
        return Point(a.x * cos(angle) + a.y * sin(angle), -a.x * sin(angle) + a.y
* cos(angle));
    }

    Point rotate_n(Point a, double angle)
    { // 将向量A逆时针旋转angle度
        return Point(a.x * cos(angle) - a.y * sin(angle), a.x * sin(angle) + a.y
* cos(angle));
    }

    Point turn_PPS(Point a, Point b, double theta){
        //将点A绕点B顺时针旋转theta(弧度)
        double x = (a.x - b.x) * cos(theta) + (a.y - b.y) * sin(theta) + b.x;
        double y = -(a.x - b.x) * sin(theta) + (a.y - b.y) * cos(theta) + b.y;
        return Point(x, y);
    }
    Point turn_PPN(Point a, Point b, double theta){
        //将点A绕点B逆时针旋转theta(弧度)
        double x = (a.x - b.x) * cos(theta) - (a.y - b.y) * sin(theta) + b.x;
        double y = (a.x - b.x) * sin(theta) + (a.y - b.y) * cos(theta) + b.y;
        return Point(x, y);
    }

    //判断两个向量是否平行
    bool checkLL(Vector a, Vector b) {
        return !sign(cross(a, b));
    }

    Point get_line_intersection(Point p, Vector v, Point q, Vector w)
```

```
    {   // 两直线的交点
        // 使用前提，直线必须有交点
        // cross(v, w) == 0则两直线平行或者重合
        Vector u = p - q;
        double t = cross(w, u) / cross(v, w);
        return p + v * t;
    }

    double distance_to_line(Point p, Point a, Point b)
    { // 点到直线的距离，直线为AB所在直线
        Vector v1 = b - a, v2 = p - a;
        return fabs(cross(v1, v2) / get_length(v1));
    }

    double distance_to_segment(Point p, Point a, Point b)
    { // 点到线段的距离，线段为线段AB
        if (a == b)
            return get_length(p - a);

        Vector v1 = b - a, v2 = p - a, v3 = p - b;
        if (sign(dot(v1, v2)) < 0)
            return get_length(v2);
        if (sign(dot(v1, v3)) > 0)
            return get_length(v3);
        return distance_to_line(p, a, b);
    }

    Point get_line_projection(Point p, Point a, Point b)
    { // 点在直线上的投影，直线为AB所在直线
        Vector v = b - a;
        return a + v * (dot(v, p - a) / dot(v, v));
    }

    bool on_segment(Point p, Point a, Point b)
    { // 点是否在线段上
        return sign(cross(p - a, p - b)) == 0 && sign(dot(p - a, p - b)) <= 0;
    }

    bool segment_intersection(Point a1, Point a2, Point b1, Point b2)
    { // 判断两个线段是否相交
        double c1 = cross(a2 - a1, b1 - a1), c2 = cross(a2 - a1, b2 - a1);
        double c3 = cross(b2 - b1, a2 - b1), c4 = cross(b2 - b1, a1 - b1);
        return sign(c1) * sign(c2) <= 0 && sign(c3) * sign(c4) <= 0;
    }
    // 多边形
    double polygon_area(Point p[], int n)
    { // 求多边形面积
        double s = 0;
        for (int i = 1; i + 1 < n; i++)
            s += cross(p[i] - p[0], p[i + 1] - p[i]);
        return s / 2;
    }
}
using namespace Geometry;
```